

OpenShift Container Platform 4.19

Architecture

An overview of the architecture for OpenShift Container Platform

Last Updated: 2026-10-06

OpenShift Container Platform 4.19 Architecture

An overview of the architecture for OpenShift Container Platform

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document provides an overview of the platform and application architecture in OpenShift Container Platform.

Table of Contents

CHAPTER 1. ARCHITECTURE OVERVIEW 4
1.1. GLOSSARY OF COMMON TERMS FOR OPENSHIFT CONTAINER PLATFORM ARCHITECTURE 4
1.2. ABOUT INSTALLATION AND UPDATES 8
1.3. ABOUT THE CONTROL PLANE 8
1.4. ABOUT CONTAINERIZED APPLICATIONS FOR DEVELOPERS 8
1.5. ABOUT RED HAT ENTERPRISE LINUX COREOS (RHCOS) AND IGNITION 9
1.6. ABOUT ADMISSION PLUGINS 9
1.7. ABOUT LINUX CGROUP VERSION 2 9

CHAPTER 2. OPENSHIFT CONTAINER PLATFORM ARCHITECTURE 11
2.1. INTRODUCTION TO OPENSHIFT CONTAINER PLATFORM 11
2.1.1. About Kubernetes 12
2.1.2. The benefits of containerized applications 13
2.1.2.1. Operating system benefits 13
2.1.2.2. Deployment and scaling benefits 13
2.1.3. OpenShift Container Platform overview 13
2.1.3.1. Custom operating system 14
2.1.3.2. Simplified installation and update process 14
2.1.3.3. Other key features 15
2.1.3.4. OpenShift Container Platform lifecycle 15
2.1.4. Internet access for OpenShift Container Platform 16

CHAPTER 3. INSTALLATION AND UPDATE 17
3.1. ABOUT OPENSHIFT CONTAINER PLATFORM INSTALLATION 17
3.1.1. About the installation program 17
3.1.2. About Red Hat Enterprise Linux CoreOS (RHCOS) 18
3.1.3. Supported platforms for OpenShift Container Platform clusters 18
3.1.4. Installation process 20
3.1.4.1. The installation process with the Assisted Installer 21
3.1.4.2. The installation process with Agent-based infrastructure 21
3.1.4.3. The installation process with installer-provisioned infrastructure 22
3.1.4.4. The installation process with user-provisioned infrastructure 22
3.1.4.5. Installation process details 22
3.1.5. Installation scope 24
3.2. ABOUT THE OPENSHIFT UPDATE SERVICE 24
3.3. SUPPORT POLICY FOR UNMANAGED OPERATORS 26
3.4. NEXT STEPS 27

CHAPTER 4. RED HAT OPENSHIFT CLUSTER MANAGER 28
4.1. ACCESSING RED HAT OPENSHIFT CLUSTER MANAGER 28
4.2. GENERAL ACTIONS 28
4.3. CLUSTER TABS 29
4.3.1. Overview tab 29
4.3.2. Access control tab 30
4.3.2.1. Identity providers 30
4.3.2.2. Cluster roles and acess 30
4.3.2.3. OCM roles and access 30
4.3.3. Add-ons tab 31
4.3.4. Insights Advisor tab 31
4.3.5. Machine pools tab 31
4.3.6. Support tab 31
4.3.7. Settings tab 31

4.4. ADDITIONAL RESOURCES 31

CHAPTER 5. ABOUT THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR 32
5.1. CLUSTER MANAGEMENT WITH MULTICLUSTER ENGINE ON OPENSHIFT CONTAINER PLATFORM 32
5.2. CLUSTER MANAGEMENT WITH RED HAT ADVANCED CLUSTER MANAGEMENT 32
5.3. ADDITIONAL RESOURCES 32

CHAPTER 6. CONTROL PLANE ARCHITECTURE 33
6.1. NODE CONFIGURATION MANAGEMENT WITH MACHINE CONFIG POOLS 33
6.2. MACHINE ROLES IN OPENSHIFT CONTAINER PLATFORM 34
6.2.1. Control plane and node host compatibility 34
6.2.2. Cluster workers 35
6.2.3. Cluster control planes 35
6.3. OPERATORS IN OPENSHIFT CONTAINER PLATFORM 37
6.3.1. Cluster Operators 38
6.3.2. Add-on Operators 38
6.4. OVERVIEW OF ETCD 39
6.4.1. Benefits of using etcd 39
6.4.2. How etcd works 39

CHAPTER 7. UNDERSTANDING OPENSHIFT CONTAINER PLATFORM DEVELOPMENT 40
7.1. ABOUT DEVELOPING CONTAINERIZED APPLICATIONS 40
7.2. BUILDING A SIMPLE CONTAINER 40
7.2.1. Container build tool options 41
7.2.2. Base image options 42
7.2.3. Registry options 43
7.3. CREATING A KUBERNETES MANIFEST FOR OPENSHIFT CONTAINER PLATFORM 43
7.3.1. About Kubernetes pods and services 44
7.3.2. Application types 44
7.3.3. Available supporting components 45
7.3.4. Applying the manifest 45
7.3.5. Next steps 46
7.4. DEVELOP FOR OPERATORS 46

CHAPTER 8. RED HAT ENTERPRISE LINUX COREOS (RHCOS) 47
8.1. ABOUT RHCOS 47
8.1.1. Key RHCOS features 47
8.1.2. Choosing how to configure RHCOS 48
8.1.3. Choosing how to deploy RHCOS 49
8.1.4. About Ignition 50
8.1.4.1. How Ignition works 50
8.1.4.2. The Ignition sequence 51
8.2. VIEWING IGNITION CONFIGURATION FILES 51
8.3. CHANGING IGNITION CONFIGS AFTER INSTALLATION 53

CHAPTER 9. ADMISSION PLUGINS 55
9.1. ABOUT ADMISSION PLUGINS 55
9.2. DEFAULT ADMISSION PLUGINS 55
9.3. WEBHOOK ADMISSION PLUGINS 58
9.4. TYPES OF WEBHOOK ADMISSION PLUGINS 59
9.4.1. Mutating admission plugin 59
9.4.2. Validating admission plugin 61
9.5. CONFIGURING DYNAMIC ADMISSION 62
9.6. ADDITIONAL RESOURCES 69

CHAPTER 1. ARCHITECTURE OVERVIEW

OpenShift Container Platform is a cloud-based Kubernetes container platform. The foundation of OpenShift Container Platform is based on Kubernetes and therefore shares the same technology. To learn more about OpenShift Container Platform and Kubernetes, see product architecture.

1.1. GLOSSARY OF COMMON TERMS FOR OPENSHIFT CONTAINER PLATFORM ARCHITECTURE

This glossary defines common terms that are used in the architecture content.

access policies

A set of roles that dictate how users, applications, and entities within a cluster interact with one another. An access policy increases cluster security.

admission plugins

Admission plugins enforce security policies, resource limitations, or configuration requirements.

authentication

To control access to an OpenShift Container Platform cluster, a cluster administrator can configure user authentication to ensure only approved users access the cluster. To interact with an OpenShift Container Platform cluster, you must authenticate with the OpenShift Container Platform API. You can authenticate by providing an OAuth access token or an X.509 client certificate in your requests to the OpenShift Container Platform API.

bootstrap

A temporary machine that runs minimal Kubernetes and deploys the OpenShift Container Platform control plane.

certificate signing requests (CSRs)

A resource requests a denoted signer to sign a certificate. This request might get approved or denied.

Cluster Version Operator (CVO)

An Operator that checks with the OpenShift Container Platform Update Service to see the valid updates and update paths based on current component versions and information in the graph.

compute nodes

Nodes that are responsible for executing workloads for cluster users. Compute nodes are also known as worker nodes.

configuration drift

A situation where the configuration on a node does not match what the machine config specifies.

containers

Lightweight and executable images that consist of software and all of its dependencies. Because containers virtualize the operating system, you can run containers anywhere, such as data centers, public or private clouds, and local hosts.

container orchestration engine

Software that automates the deployment, management, scaling, and networking of containers.

container workloads

Applications that are packaged and deployed in containers.

control groups (cgroups)

Partitions sets of processes into groups to manage and limit the resources processes consume.

control plane

A container orchestration layer that exposes the API and interfaces to define, deploy, and manage the life cycle of containers. Control planes are also known as control plane machines.

CRI-O

A Kubernetes native container runtime implementation that integrates with the operating system to deliver an efficient Kubernetes experience.

deployment

A Kubernetes resource object that maintains the life cycle of an application.

Dockerfile

A text file that contains the user commands to perform on a terminal to assemble the image.

hosted control planes

A OpenShift Container Platform feature that enables hosting a control plane on the OpenShift Container Platform cluster from its data plane and workers. This model performs the following actions:

- Optimize infrastructure costs required for the control planes.
- Improve the cluster creation time.
- Enable hosting the control plane using the Kubernetes native high level primitives. For example, deployments and stateful sets.
- Allow a strong network segmentation between the control plane and workloads.

hybrid cloud deployments

Deployments that deliver a consistent platform across bare metal, virtual, private, and public cloud environments. This offers speed, agility, and portability.

Ignition

A utility that RHCOS uses to manipulate disks during initial configuration. It completes common disk tasks, including partitioning disks, formatting partitions, writing files, and configuring users.

installer-provisioned infrastructure

The installation program deploys and configures the infrastructure that the cluster runs on.

kubelet

A primary node agent that runs on each node in the cluster to ensure that containers are running in a pod.

kubernetes manifest

Specifications of a Kubernetes API object in a JSON or YAML format. A configuration file can include deployments, config maps, secrets, daemon sets.

Machine Config Daemon (MCD)

A daemon that regularly checks the nodes for configuration drift.

Machine Config Operator (MCO)

An Operator that applies the new configuration to your cluster machines.

machine config pools (MCP)

A group of machines, such as control plane components or user workloads, that are based on the resources that they handle.

metadata

Additional information about cluster deployment artifacts.

microservices

An approach to writing software. Applications can be separated into the smallest components, independent from each other by using microservices.

mirror registry

A registry that holds the mirror of OpenShift Container Platform images.

monolithic applications

Applications that are self-contained, built, and packaged as a single piece.

namespaces

A namespace isolates specific system resources that are visible to all processes. Inside a namespace, only processes that are members of that namespace can see those resources.

networking

Network information of OpenShift Container Platform cluster.

node

A worker machine in the OpenShift Container Platform cluster. A node is either a virtual machine (VM) or a physical machine.

OpenShift CLI (oc)

A command-line tool to run OpenShift Container Platform commands on the terminal.

OpenShift Dedicated

A managed RHEL OpenShift Container Platform offering on Amazon Web Services (AWS) and Google Cloud. OpenShift Dedicated focuses on building and scaling applications.

OpenShift Update Service (OSUS)

For clusters with internet access, Red Hat Enterprise Linux (RHEL) provides over-the-air updates by using an OpenShift update service as a hosted service located behind public APIs.

OpenShift image registry

A registry provided by OpenShift Container Platform to manage images.

Operator

The preferred method of packaging, deploying, and managing a Kubernetes application in an OpenShift Container Platform cluster. An Operator takes human operational knowledge and encodes it into software that is packaged and shared with customers.

OperatorHub

A platform that contains various OpenShift Container Platform Operators to install.

Operator Lifecycle Manager (OLM)

OLM helps you to install, update, and manage the lifecycle of Kubernetes native applications. OLM is an open source toolkit designed to manage Operators in an effective, automated, and scalable way.

OSTree

An upgrade system for Linux-based operating systems that performs atomic upgrades of complete file system trees. OSTree tracks meaningful changes to the file system tree using an addressable object store, and is designed to complement existing package management systems.

over-the-air (OTA) updates

The OpenShift Container Platform Update Service (OSUS) provides over-the-air updates to OpenShift Container Platform, including Red Hat Enterprise Linux CoreOS (RHCOS).

pod

One or more containers with shared resources, such as volume and IP addresses, running in your OpenShift Container Platform cluster. A pod is the smallest compute unit defined, deployed, and managed.

private registry

OpenShift Container Platform can use any server implementing the container image registry API as a source of the image which allows the developers to push and pull their private container images.

public registry

OpenShift Container Platform can use any server implementing the container image registry API as a source of the image which allows the developers to push and pull their public container images.

RHEL OpenShift Container Platform Cluster Manager

A managed service where you can install, modify, operate, and upgrade your OpenShift Container Platform clusters.

RHEL Quay Container Registry

A Quay.io container registry that serves most of the container images and Operators to OpenShift Container Platform clusters.

replication controllers

An asset that indicates how many pod replicas are required to run at a time.

role-based access control (RBAC)

A key security control to ensure that cluster users and workloads have only access to resources required to execute their roles.

route

Routes expose a service to allow for network access to pods from users and applications outside the OpenShift Container Platform instance.

scaling

The increasing or decreasing of resource capacity.

service

A service exposes a running application on a set of pods.

Source-to-Image (S2I) image

An image created based on the programming language of the application source code in OpenShift Container Platform to deploy applications.

storage

OpenShift Container Platform supports many types of storage, both for on-premise and cloud providers. You can manage container storage for persistent and non-persistent data in an OpenShift Container Platform cluster.

Telemetry

A component to collect information such as size, health, and status of OpenShift Container Platform.

template

A template describes a set of objects that can be parameterized and processed to produce a list of objects for creation by OpenShift Container Platform.

user-provisioned infrastructure

You can install OpenShift Container Platform on the infrastructure that you provide. You can use the installation program to generate the assets required to provision the cluster infrastructure, create the cluster infrastructure, and then deploy the cluster to the infrastructure that you provided.

web console

A user interface (UI) to manage OpenShift Container Platform.

worker node

Nodes that are responsible for executing workloads for cluster users. Worker nodes are also known as compute nodes.

Additional resources

- For more information on networking, see OpenShift Container Platform networking.
- For more information on storage, see OpenShift Container Platform storage.
- For more information on authentication, see OpenShift Container Platform authentication.
- For more information on Operator Lifecycle Manager (OLM), see OLM.
- For more information on over-the-air (OTA) updates, see Introduction to OpenShift updates.

1.2. ABOUT INSTALLATION AND UPDATES

As a cluster administrator, you can use the OpenShift Container Platform installation program to install and deploy a cluster by using one of the following methods:

- Installer-provisioned infrastructure
- User-provisioned infrastructure

1.3. ABOUT THE CONTROL PLANE

The control plane manages the worker nodes and the pods in your cluster. You can configure nodes with the use of machine config pools (MCPs). MCPs are groups of machines, such as control plane components or user workloads, that are based on the resources that they handle. OpenShift Container Platform assigns different roles to hosts. These roles define the function of a machine in a cluster. The cluster contains definitions for the standard control plane and worker role types.

You can use Operators to package, deploy, and manage services on the control plane. Operators are important components in OpenShift Container Platform because they provide the following services:

- Perform health checks
- Provide ways to watch applications
- Manage over-the-air updates
- Ensure applications stay in the specified state

Additional resources

- Hosted control planes overview

1.4. ABOUT CONTAINERIZED APPLICATIONS FOR DEVELOPERS

As a developer, you can use different tools, methods, and formats to develop your containerized application based on your unique requirements, for example:

- Use various build-tool, base-image, and registry options to build a simple container application.
- Use supporting components such as OperatorHub and templates to develop your application.
- Package and deploy your application as an Operator.

You can also create a Kubernetes manifest and store it in a Git repository. Kubernetes works on basic units called pods. A pod is a single instance of a running process in your cluster. Pods can contain one or more containers. You can create a service by grouping a set of pods and their access policies. Services provide permanent internal IP addresses and host names for other applications to use as pods are created and destroyed. Kubernetes defines workloads based on the type of your application.

1.5. ABOUT RED HAT ENTERPRISE LINUX COREOS (RHCOS) AND IGNITION

As a cluster administrator, you can perform the following Red Hat Enterprise Linux CoreOS (RHCOS) tasks:

- Learn about the next generation of single-purpose container operating system technology.
- Choose how to configure Red Hat Enterprise Linux CoreOS (RHCOS)
- Choose how to deploy Red Hat Enterprise Linux CoreOS (RHCOS):
  - Installer-provisioned deployment
  - User-provisioned deployment

The OpenShift Container Platform installation program creates the Ignition configuration files that you need to deploy your cluster. Red Hat Enterprise Linux CoreOS (RHCOS) uses Ignition during the initial configuration to perform common disk tasks, such as partitioning, formatting, writing files, and configuring users. During the first boot, Ignition reads its configuration from the installation media or the location that you specify and applies the configuration to the machines.

You can learn how Ignition works, the process for a Red Hat Enterprise Linux CoreOS (RHCOS) machine in an OpenShift Container Platform cluster, view Ignition configuration files, and change Ignition configuration after an installation.

1.6. ABOUT ADMISSION PLUGINS

You can use admission plugins to regulate how OpenShift Container Platform functions. After a resource request is authenticated and authorized, admission plugins intercept the resource request to the master API to validate resource requests and to ensure that scaling policies are adhered to. Admission plugins are used to enforce security policies, resource limitations, configuration requirements, and other settings.

1.7. ABOUT LINUX CGROUP VERSION 2

OpenShift Container Platform uses Linux control group version 2 (cgroup v2) in your cluster.

cgroup v2 offers several improvements over cgroup v1, including a unified hierarchy, safer sub-tree delegation, features such as Pressure Stall Information, and enhanced resource management and isolation. However, cgroup v2 has different CPU, memory, and I/O management characteristics than cgroup v1. Therefore, some workloads might experience slight differences in memory or CPU usage on clusters that run cgroup v2.



NOTE

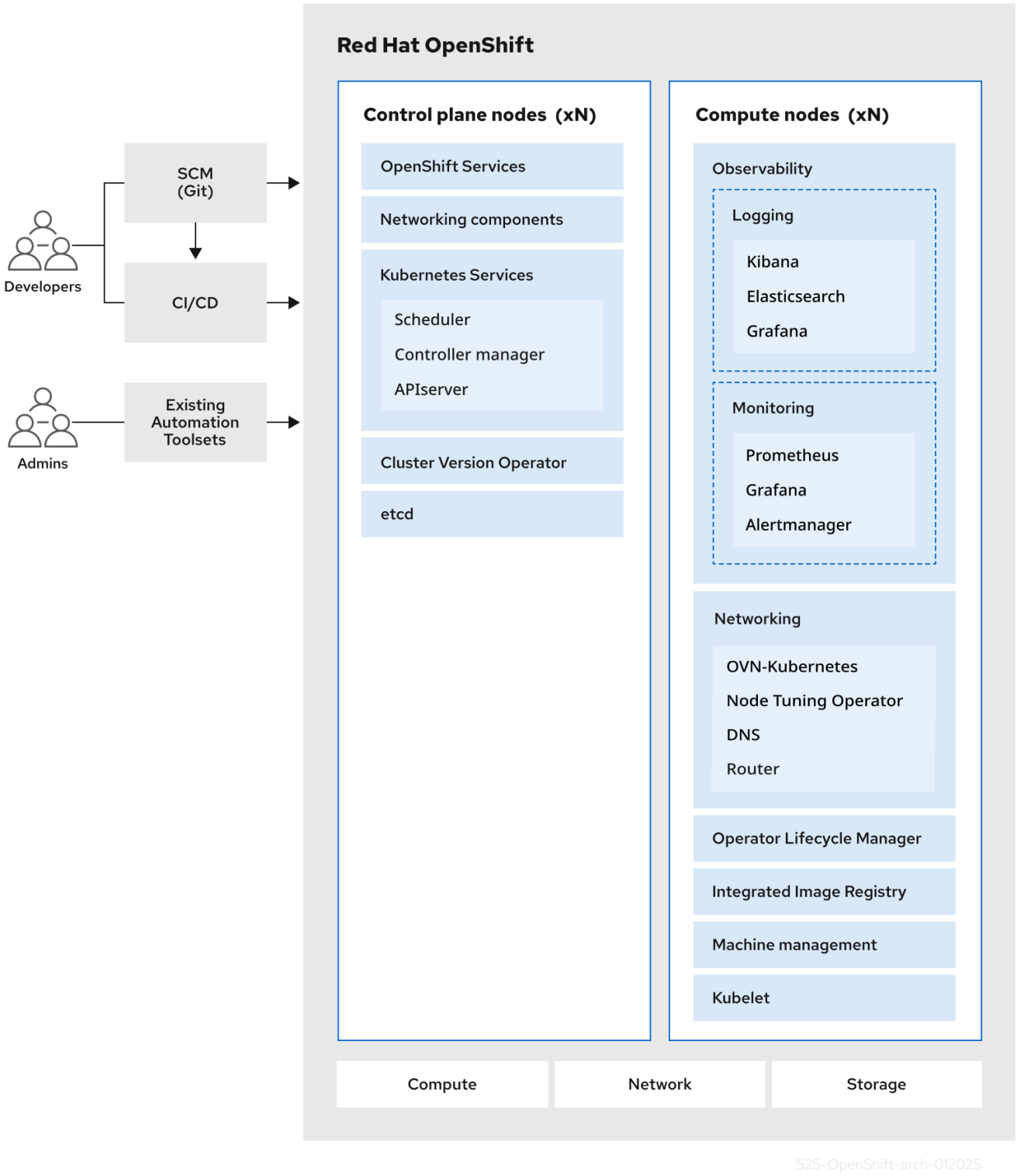
- If you run third-party monitoring and security agents that depend on the cgroup file system, update the agents to a version that supports cgroup v2.
- If you have configured cgroup v2 and run cAdvisor as a stand-alone daemon set for monitoring pods and containers, update cAdvisor to v0.43.0 or later.
- If you deploy Java applications, use versions that fully support cgroup v2, such as the following packages:
  - OpenJDK / HotSpot: jdk8u372, 11.0.16, 15 and later
  - NodeJs 20.3.0 and later
  - IBM Semeru Runtimes: jdk8u345-b01, 11.0.16.0, 17.0.4.0, 18.0.2.0 and later
  - IBM SDK Java Technology Edition Version (IBM Java): 8.0.7.15 and later

CHAPTER 2. OPENSHIFT CONTAINER PLATFORM ARCHITECTURE

2.1. INTRODUCTION TO OPENSHIFT CONTAINER PLATFORM

OpenShift Container Platform is a platform for developing and running containerized applications. It is designed to allow applications and the data centers that support them to expand from just a few machines and applications to thousands of machines that serve millions of clients.

With its foundation in Kubernetes, OpenShift Container Platform incorporates the same technology that serves as the engine for massive telecommunications, streaming video, gaming, banking, and other applications. Its implementation in open Red Hat technologies lets you extend your containerized applications beyond a single cloud to on-premise and multi-cloud environments.



2.1.1. About Kubernetes

Although container images and the containers that run from them are the primary building blocks for modern application development, to run them at scale requires a reliable and flexible distribution system. Kubernetes is the defacto standard for orchestrating containers.

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The general concept of Kubernetes is fairly simple:

- Start with one or more worker nodes to run the container workloads.

- Manage the deployment of those workloads from one or more control plane nodes.
- Wrap containers in a deployment unit called a pod. Using pods provides extra metadata with the container and offers the ability to group several containers in a single deployment entity.
- Create special kinds of assets. For example, services are represented by a set of pods and a policy that defines how they are accessed. This policy allows containers to connect to the services that they need even if they do not have the specific IP addresses for the services. Replication controllers are another special asset that indicates how many pod replicas are required to run at a time. You can use this capability to automatically scale your application to adapt to its current demand.

In only a few years, Kubernetes has seen massive cloud and on-premise adoption. The open source development model allows many people to extend Kubernetes by implementing different technologies for components such as networking, storage, and authentication.

2.1.2. The benefits of containerized applications

Using containerized applications offers many advantages over using traditional deployment methods. Where applications were once expected to be installed on operating systems that included all their dependencies, containers let an application carry their dependencies with them. Creating containerized applications offers many benefits.

2.1.2.1. Operating system benefits

Containers use small, dedicated Linux operating systems without a kernel. Their file system, networking, cgroups, process tables, and namespaces are separate from the host Linux system, but the containers can integrate with the hosts seamlessly when necessary. Being based on Linux allows containers to use all the advantages that come with the open source development model of rapid innovation.

Because each container uses a dedicated operating system, you can deploy applications that require conflicting software dependencies on the same host. Each container carries its own dependent software and manages its own interfaces, such as networking and file systems, so applications never need to compete for those assets.

2.1.2.2. Deployment and scaling benefits

If you employ rolling upgrades between major releases of your application, you can continuously improve your applications without downtime and still maintain compatibility with the current release.

You can also deploy and test a new version of an application alongside the existing version. If the container passes your tests, simply deploy more new containers and remove the old ones.

Since all the software dependencies for an application are resolved within the container itself, you can use a standardized operating system on each host in your data center. You do not need to configure a specific operating system for each application host. When your data center needs more capacity, you can deploy another generic host system.

Similarly, scaling containerized applications is simple. OpenShift Container Platform offers a simple, standard way of scaling any containerized service. For example, if you build applications as a set of microservices rather than large, monolithic applications, you can scale the individual microservices individually to meet demand. This capability allows you to scale only the required services instead of the entire application, which can allow you to meet application demands while using minimal resources.

2.1.3. OpenShift Container Platform overview

OpenShift Container Platform provides enterprise-ready enhancements to Kubernetes, including the following enhancements:

- Hybrid cloud deployments. You can deploy OpenShift Container Platform clusters to a variety of public cloud platforms or in your data center.
- Integrated Red Hat technology. Major components in OpenShift Container Platform come from Red Hat Enterprise Linux (RHEL) and related Red Hat technologies. OpenShift Container Platform benefits from the intense testing and certification initiatives for Red Hat's enterprise quality software.
- Open source development model. Development is completed in the open, and the source code is available from public software repositories. This open collaboration fosters rapid innovation and development.

Although Kubernetes excels at managing your applications, it does not specify or manage platform-level requirements or deployment processes. Powerful and flexible platform management tools and processes are important benefits that OpenShift Container Platform 4.19 offers. The following sections describe some unique features and benefits of OpenShift Container Platform.

2.1.3.1. Custom operating system

OpenShift Container Platform uses Red Hat Enterprise Linux CoreOS (RHCOS), a container-oriented operating system that is specifically designed for running containerized applications from OpenShift Container Platform and works with new tools to provide fast installation, Operator-based management, and simplified upgrades.

RHCOS includes:

- Ignition, which OpenShift Container Platform uses as a firstboot system configuration for initially bringing up and configuring machines.
- CRI-O, a Kubernetes native container runtime implementation that integrates closely with the operating system to deliver an efficient and optimized Kubernetes experience. CRI-O provides facilities for running, stopping, and restarting containers. It fully replaces the Docker Container Engine, which was used in OpenShift Container Platform 3.
- Kubelet, the primary node agent for Kubernetes that is responsible for launching and monitoring containers.

In OpenShift Container Platform 4.19, you must use RHCOS for all control plane machines, but you can use Red Hat Enterprise Linux (RHEL) as the operating system for compute machines, which are also known as worker machines. If you choose to use RHEL workers, you must perform more system maintenance than if you use RHCOS for all of the cluster machines.

2.1.3.2. Simplified installation and update process

With OpenShift Container Platform 4.19, if you have an account with the right permissions, you can deploy a production cluster in supported clouds by running a single command and providing a few values. You can also customize your cloud installation or install your cluster in your data center if you use a supported platform.

For clusters that use RHCOS for all machines, updating, or upgrading, OpenShift Container Platform is a simple, highly-automated process. Because OpenShift Container Platform completely controls the systems and services that run on each machine, including the operating system itself, from a central

control plane, upgrades are designed to become automatic events. If your cluster contains RHEL worker machines, the control plane benefits from the streamlined update process, but you must perform more tasks to upgrade the RHEL machines.

2.1.3.3. Other key features

Operators are both the fundamental unit of the OpenShift Container Platform 4.19 code base and a convenient way to deploy applications and software components for your applications to use. In OpenShift Container Platform, Operators serve as the platform foundation and remove the need for manual upgrades of operating systems and control plane applications. OpenShift Container Platform Operators such as the Cluster Version Operator and Machine Config Operator allow simplified, cluster-wide management of those critical components.

Operator Lifecycle Manager (OLM) and the OperatorHub provide facilities for storing and distributing Operators to people developing and deploying applications.

The Red Hat Quay Container Registry is a Quay.io container registry that serves most of the container images and Operators to OpenShift Container Platform clusters. Quay.io is a public registry version of Red Hat Quay that stores millions of images and tags.

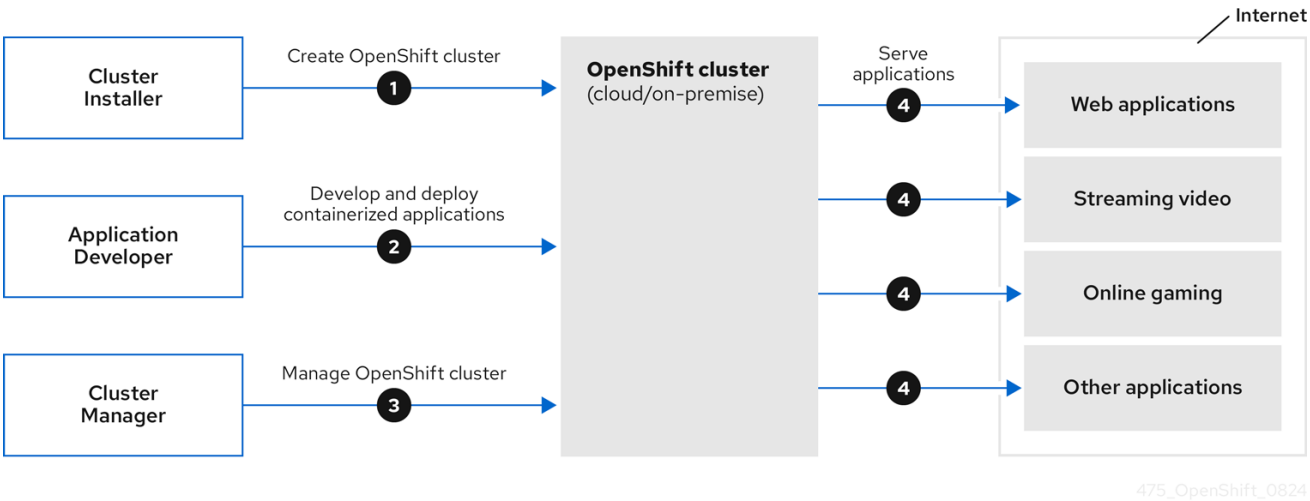
Other enhancements to Kubernetes in OpenShift Container Platform include improvements in software defined networking (SDN), authentication, log aggregation, monitoring, and routing. OpenShift Container Platform also offers a comprehensive web console and the custom OpenShift CLI (**oc**) interface.

2.1.3.4. OpenShift Container Platform lifecycle

The following figure illustrates the basic OpenShift Container Platform lifecycle:

- Creating an OpenShift Container Platform cluster
- Managing the cluster
- Developing and deploying applications
- Scaling up applications

Figure 2.1. High level OpenShift Container Platform overview

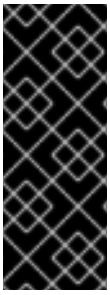


2.1.4. Internet access for OpenShift Container Platform

In OpenShift Container Platform 4.19, you require access to the internet to install your cluster.

You must have internet access to perform the following actions:

- Access OpenShift Cluster Manager to download the installation program and perform subscription management. If the cluster has internet access and you do not disable Telemetry, that service automatically entitles your cluster.
- Access Quay.io to obtain the packages that are required to install your cluster.
- Obtain the packages that are required to perform cluster updates.



IMPORTANT

If your cluster cannot have direct internet access, you can perform a restricted network installation on some types of infrastructure that you provision. During that process, you download the required content and use it to populate a mirror registry with the installation packages. With some installation types, the environment that you install your cluster in will not require internet access. Before you update the cluster, you update the content of the mirror registry.

CHAPTER 3. INSTALLATION AND UPDATE

3.1. ABOUT OPENSHIFT CONTAINER PLATFORM INSTALLATION

The OpenShift Container Platform installation program offers four methods for deploying a cluster which are detailed in the following list:

- **Interactive**: You can deploy a cluster with the web-based Assisted Installer. This is an ideal approach for clusters with networks connected to the internet. The Assisted Installer is the easiest way to install OpenShift Container Platform, it provides smart defaults, and it performs pre-flight validations before installing the cluster. It also provides a RESTful API for automation and advanced configuration scenarios.

- **Local Agent-based**: You can deploy a cluster locally with the Agent-based Installer for disconnected environments or restricted networks. It provides many of the benefits of the Assisted Installer, but you must download and configure the Agent-based Installer first. Configuration is done with a command-line interface. This approach is ideal for disconnected environments.

- **Automated**: You can deploy a cluster on installer-provisioned infrastructure. The installation program uses each cluster host's baseboard management controller (BMC) for provisioning. You can deploy clusters in connected or disconnected environments.

- **Full control**: You can deploy a cluster on infrastructure that you prepare and maintain, which provides maximum customizability. You can deploy clusters in connected or disconnected environments.

Each method deploys a cluster with the following characteristics:

- Highly available infrastructure with no single points of failure, which is available by default.

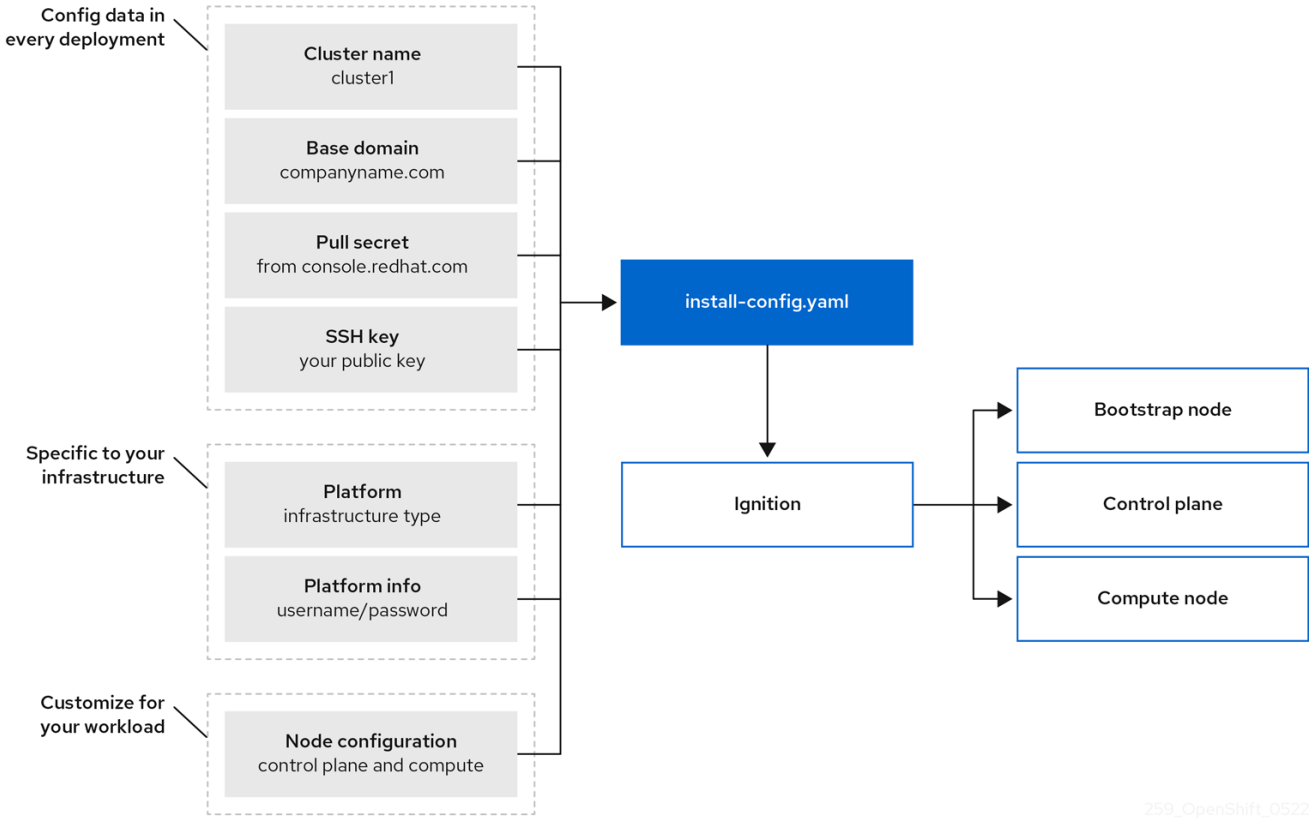
- Administrators can control what updates are applied and when.

3.1.1. About the installation program

You can use the installation program to deploy each type of cluster. The installation program generates the main assets, such as Ignition config files for the bootstrap, control plane, and compute machines. You can start an OpenShift Container Platform cluster with these three machine configurations, provided you correctly configured the infrastructure.

The OpenShift Container Platform installation program uses a set of targets and dependencies to manage cluster installations. The installation program has a set of targets that it must achieve, and each target has a set of dependencies. Because each target is only concerned with its own dependencies, the installation program can act to achieve multiple targets in parallel with the ultimate target being a running cluster. The installation program recognizes and uses existing components instead of running commands to create them again because the program meets the dependencies.

Figure 3.1. OpenShift Container Platform installation targets and dependencies



3.1.2. About Red Hat Enterprise Linux CoreOS (RHCOS)

Post-installation, each cluster machine uses Red Hat Enterprise Linux CoreOS (RHCOS) as the operating system. RHCOS is the immutable container host version of Red Hat Enterprise Linux (RHEL) and features a RHEL kernel with SELinux enabled by default. RHCOS includes the **kubelet**, which is the Kubernetes node agent, and the CRI-O container runtime, which is optimized for Kubernetes.

Every control plane machine in an OpenShift Container Platform 4.19 cluster must use RHCOS, which includes a critical first-boot provisioning tool called Ignition. This tool enables the cluster to configure the machines. Operating system updates are delivered as a bootable container image, using **OSTree** as a backend, that is deployed across the cluster by the Machine Config Operator. Actual operating system changes are made in-place on each machine as an atomic operation by using **rpm-ostree**. Together, these technologies enable OpenShift Container Platform to manage the operating system like it manages any other application on the cluster, by in-place upgrades that keep the entire platform up to date. These in-place updates can reduce the burden on operations teams.

If you use RHCOS as the operating system for all cluster machines, the cluster manages all aspects of its components and machines, including the operating system. Because of this, only the installation program and the Machine Config Operator can change machines. The installation program uses Ignition config files to set the exact state of each machine, and the Machine Config Operator completes more changes to the machines, such as the application of new certificates or keys, after installation.

3.1.3. Supported platforms for OpenShift Container Platform clusters

The following table describes which platforms are supported by the different methods available for installing OpenShift Container Platform clusters:

Table 3.1. Supported platforms

Platform	Installer-provisioned infrastructure [1]	User-provisioned infrastructure [2]	Agent-based Installer	Assisted Installer
Amazon Web Services (AWS)	X	X		
Bare metal	X	X	X	X
External			X	X
Google Cloud	X	X		
IBM Cloud® Classic	X			
IBM Cloud® Virtual Private Cloud (VPC)	X			
IBM Power®		X	X	X
IBM Z® or IBM® LinuxONE		X	X	X
Microsoft Azure	X	X		
Microsoft Azure Stack Hub	X	X		
None			X	X
Nutanix	X			X
Oracle Cloud Infrastructure (OCI)			X	X
Red Hat OpenStack Platform (RHOSP) [3]	X	X		
VMware vSphere	X	X	X	X

1. For installer-provisioned infrastructure: All machines, including the computer that you run the installation process on, must have direct internet access to pull images for platform containers and provide telemetry data to Red Hat.



IMPORTANT

After installation, the following changes are not supported:

- Mixing cloud provider platforms.
- Mixing cloud provider components. For example, using a persistent storage framework from a another platform on the platform where you installed the cluster.

2. For user-provisioned infrastructure: Depending on the supported cases for the platform, you can perform installations on user-provisioned infrastructure so that you can run machines with full internet access, place your cluster behind a proxy, or perform a disconnected installation. In a disconnected installation, you can download the images that are required to install a cluster, place them in a mirror registry, and use that data to install your cluster. While you require internet access to pull images for platform containers, with a disconnected installation on vSphere or bare-metal infrastructure, your cluster machines do not require direct internet access.

3. For Red Hat OpenStack Platform (RHOSP): The latest OpenShift Container Platform release supports both the latest RHOSP long-life release and intermediate release. For complete RHOSP release compatibility, see the OpenShift Container Platform on RHOSP support matrix.

The OpenShift Container Platform 4.x Tested Integrations page contains details about integration testing for different platforms.

3.1.4. Installation process

Except for the Assisted Installer, when you install an OpenShift Container Platform cluster, you must download the installation program from the appropriate **Cluster Type** page on the OpenShift Cluster Manager Hybrid Cloud Console. This console manages:

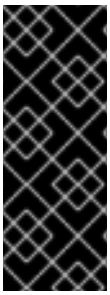
- REST API for accounts.
- Registry tokens, which are the pull secrets that you use to obtain the required components.
- Cluster registration, which associates the cluster identity to your Red Hat account to facilitate the gathering of usage metrics.

In OpenShift Container Platform 4.19, the installation program is a Go binary file that performs a series of file transformations on a set of assets. The way you interact with the installation program differs depending on your installation type. Consider the following installation use cases:

- To deploy a cluster with the Assisted Installer, you must configure the cluster settings by using the Assisted Installer. There is no installation program to download and configure. After you finish setting the cluster configuration, you download a discovery ISO and then boot cluster machines with that image. You can install clusters with the Assisted Installer on Nutanix, vSphere, and bare metal with full integration, and other platforms without integration. If you install on bare metal, you must provide all of the cluster infrastructure and resources, including the networking, load balancing, storage, and individual cluster machines.
- To deploy clusters with the Agent-based Installer, you can download the Agent-based Installer first. You can then configure the cluster and generate a discovery image. You boot cluster machines with the discovery image, which installs an agent that communicates with the installation program and handles the provisioning for you instead of you interacting with the installation program or setting up a provisioner machine yourself. You must provide all of the cluster infrastructure and resources, including the networking, load balancing, storage, and individual cluster machines. This approach is ideal for disconnected environments.
- For clusters with installer-provisioned infrastructure, you delegate the infrastructure bootstrapping and provisioning to the installation program instead of doing it yourself. The installation program creates all of the networking, machines, and operating systems that are required to support the cluster, except if you install on bare metal. If you install on bare metal, you must provide all of the cluster infrastructure and resources, including the bootstrap machine, networking, load balancing, storage, and individual cluster machines.

- If you provision and manage the infrastructure for your cluster, you must provide all of the cluster infrastructure and resources, including the bootstrap machine, networking, load balancing, storage, and individual cluster machines.

For the installation program, the program uses three sets of files during installation: an installation configuration file that is named **install-config.yaml**, Kubernetes manifests, and Ignition config files for your machine types.

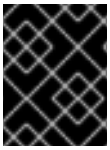


IMPORTANT

You can modify Kubernetes and the Ignition config files that control the underlying RHCOS operating system during installation. However, no validation is available to confirm the suitability of any modifications that you make to these objects. If you modify these objects, you might render your cluster non-functional. Because of this risk, modifying Kubernetes and Ignition config files is not supported unless you are following documented procedures or are instructed to do so by Red Hat support.

The installation configuration file is transformed into Kubernetes manifests, and then the manifests are wrapped into Ignition config files. The installation program uses these Ignition config files to create the cluster.

The installation configuration files are all pruned when you run the installation program, so be sure to back up all the configuration files that you want to use again.



IMPORTANT

You cannot modify the parameters that you set during installation, but you can modify many cluster attributes after installation.

3.1.4.1. The installation process with the Assisted Installer

Installation with the Assisted Installer involves creating a cluster configuration interactively by using the web-based user interface or the RESTful API. The Assisted Installer user interface prompts you for required values and provides reasonable default values for the remaining parameters, unless you change them in the user interface or with the API. The Assisted Installer generates a discovery image, which you download and use to boot the cluster machines. The image installs RHCOS and an agent, and the agent handles the provisioning for you. You can install OpenShift Container Platform with the Assisted Installer and full integration on Nutanix, vSphere, and bare metal. Additionally, you can install OpenShift Container Platform with the Assisted Installer on other platforms without integration.

OpenShift Container Platform manages all aspects of the cluster, including the operating system itself. Each machine boots with a configuration that references resources hosted in the cluster that it joins. This configuration allows the cluster to manage itself as updates are applied.

If possible, use the Assisted Installer feature to avoid having to download and configure the Agent-based Installer.

3.1.4.2. The installation process with Agent-based infrastructure

Agent-based installation is similar to using the Assisted Installer, except that you must initially download and install the Agent-based Installer. An Agent-based installation is useful when you want the convenience of the Assisted Installer, but you need to install a cluster in a disconnected environment.

If possible, use the Agent-based installation feature to avoid having to create a provisioner machine with a bootstrap VM, and then provision and maintain the cluster infrastructure.

3.1.4.3. The installation process with installer-provisioned infrastructure

The default installation type uses installer-provisioned infrastructure. By default, the installation program acts as an installation wizard, prompting you for values that it cannot determine on its own and providing reasonable default values for the remaining parameters. You can also customize the installation process to support advanced infrastructure scenarios. The installation program provisions the underlying infrastructure for the cluster.

You can install either a standard cluster or a customized cluster. With a standard cluster, you provide minimum details that are required to install the cluster. With a customized cluster, you can specify more details about the platform, such as the number of machines that the control plane uses, the type of virtual machine that the cluster deploys, or the CIDR range for the Kubernetes service network.

If possible, use this feature to avoid having to provision and maintain the cluster infrastructure. In all other environments, you use the installation program to generate the assets that you require to provision your cluster infrastructure.

With installer-provisioned infrastructure clusters, OpenShift Container Platform manages all aspects of the cluster, including the operating system itself. Each machine boots with a configuration that references resources hosted in the cluster that it joins. This configuration allows the cluster to manage itself as updates are applied.

3.1.4.4. The installation process with user-provisioned infrastructure

You can also install OpenShift Container Platform on infrastructure that you provide. You use the installation program to generate the assets that you require to provision the cluster infrastructure, create the cluster infrastructure, and then deploy the cluster to the infrastructure that you provided.

If you do not use infrastructure that the installation program provisioned, you must manage and maintain the cluster resources yourself. The following list details some of these self-managed resources:

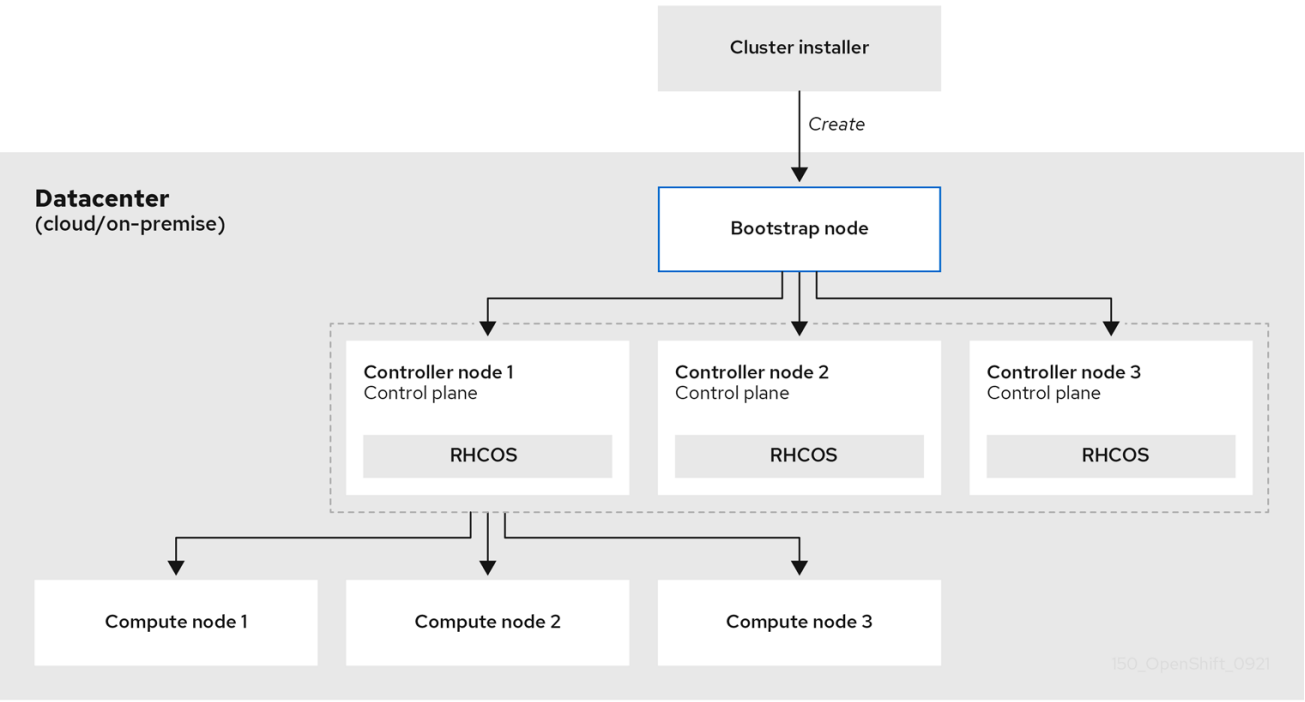
- The underlying infrastructure for the control plane and compute machines that make up the cluster
- Load balancers
- Cluster networking, including the DNS records and required subnets
- Storage for the cluster infrastructure and applications

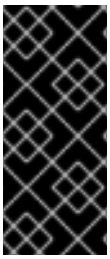
If your cluster uses user-provisioned infrastructure, you have the option of adding RHEL compute machines to your cluster.

3.1.4.5. Installation process details

When a cluster is provisioned, each machine in the cluster requires information about the cluster. OpenShift Container Platform uses a temporary bootstrap machine during initial configuration to provide the required information to the permanent control plane. The temporary bootstrap machine boots by using an Ignition config file that describes how to create the cluster. The bootstrap machine creates the control plane machines that make up the control plane. The control plane machines then create the compute machines, which are also known as worker machines. The following figure illustrates this process:

Figure 3.2. Creating the bootstrap, control plane, and compute machines





IMPORTANT

While planning to deploy your cluster, ensure that you are familiar with the recommended practices for performance and scalability, particularly the requirements for input/output (I/O) latency for etcd storage and the requirements for the recommended control plane node sizing. For more information, see “Recommended etcd practices” and “Control plane node sizing”.

After the cluster machines initialize, the bootstrap machine is destroyed. All clusters use the bootstrap process to initialize the cluster, but if you provision the infrastructure for your cluster, you must complete many of the steps manually.



IMPORTANT

- The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending **node-bootstrapper** certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.

- Consider using Ignition config files within 12 hours after they are generated, because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

Bootstrapping a cluster involves the following steps:

1. The bootstrap machine boots and starts hosting the remote resources required for the control plane machines to boot. If you provision the infrastructure, this step requires manual intervention.
2. The bootstrap machine starts a single-node etcd cluster and a temporary Kubernetes control plane.
3. The control plane machines fetch the remote resources from the bootstrap machine and finish booting. If you provision the infrastructure, this step requires manual intervention.
4. The temporary control plane schedules the production control plane to the production control plane machines.
5. The Cluster Version Operator (CVO) comes online and installs the etcd Operator. The etcd Operator scales up etcd on all control plane nodes.
6. The temporary control plane shuts down and passes control to the production control plane.
7. The bootstrap machine injects OpenShift Container Platform components into the production control plane.
8. The installation program shuts down the bootstrap machine. If you provision the infrastructure, this step requires manual intervention.
9. The control plane sets up the compute nodes.
10. The control plane installs additional services in the form of a set of Operators.

The result of this bootstrapping process is a running OpenShift Container Platform cluster. The cluster then downloads and configures remaining components needed for the day-to-day operations, including the creation of compute machines in supported environments.

Additional resources

- Recommended etcd practices
- Control plane node sizing

3.1.5. Installation scope

The scope of the OpenShift Container Platform installation program is intentionally narrow. It is designed for simplicity and ensured success. You can complete many more configuration tasks after installation completes.

Additional resources

- See Available cluster customizations for details about OpenShift Container Platform configuration resources.

3.2. ABOUT THE OPENSHIFT UPDATE SERVICE

The OpenShift Update Service (OSUS) provides update recommendations to OpenShift Container Platform, including Red Hat Enterprise Linux CoreOS (RHCOS). It provides a graph, or diagram, that contains the *vertices* of component Operators and the *edges* that connect them.

The edges in the graph show which versions you can safely update to. The vertices are update payloads that specify the intended state of the managed cluster components.

The Cluster Version Operator (CVO) in your cluster checks with the OpenShift Update Service to see the valid updates and update paths based on current component versions and information in the graph. When you request an update, the CVO uses the corresponding release image to update your cluster. The release artifacts are hosted in Quay as container images.

To allow the OpenShift Update Service to provide only compatible updates, a release verification pipeline drives automation. Each release artifact is verified for compatibility with supported cloud platforms and system architectures, as well as other component packages. After the pipeline confirms the suitability of a release, the OpenShift Update Service notifies you that it is available.

The OpenShift Update Service (OSUS) supports a single-stream release model, where only one release version is active and supported at any given time. When a new release is deployed, it fully replaces the previous release.

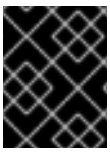
The updated release provides support for upgrades from all OpenShift Container Platform versions starting after 4.8 up to the new release version.



IMPORTANT

The OpenShift Update Service displays all recommended updates for your current cluster. If an update path is not recommended by the OpenShift Update Service, it might be because of a known issue related to the update path, such as incompatibility or availability.

Two controllers run during continuous update mode. The first controller continuously updates the payload manifests, applies the manifests to the cluster, and outputs the controlled rollout status of the Operators to indicate whether they are available, upgrading, or failed. The second controller polls the OpenShift Update Service to determine if updates are available.



IMPORTANT

Only updating to a newer version is supported. Reverting or rolling back your cluster to a previous version is not supported. If your update fails, contact Red Hat support.

During the update process, the Machine Config Operator (MCO) applies the new configuration to your cluster machines. The MCO cordons the number of nodes specified by the **maxUnavailable** field on the machine configuration pool and marks them unavailable. By default, this value is set to **1**. The MCO updates the affected nodes alphabetically by zone, based on the **topology.kubernetes.io/zone** label. If a zone has more than one node, the oldest nodes are updated first. For nodes that do not use zones, such as in bare metal deployments, the nodes are updated by age, with the oldest nodes updated first. The MCO updates the number of nodes as specified by the **maxUnavailable** field on the machine configuration pool at a time. The MCO then applies the new configuration and reboots the machine.



WARNING

The default setting for **maxUnavailable** is **1** for all the machine config pools in OpenShift Container Platform. It is recommended to not change this value and update one control plane node at a time. Do not change this value to **3** for the control plane pool.

If you use Red Hat Enterprise Linux (RHEL) machines as workers, the MCO does not update the kubelet because you must update the OpenShift API on the machines first.

With the specification for the new version applied to the old kubelet, the RHEL machine cannot return to the **Ready** state. You cannot complete the update until the machines are available. However, the maximum number of unavailable nodes is set to ensure that normal cluster operations can continue with that number of machines out of service.

The OpenShift Update Service is composed of an Operator and one or more application instances.

3.3. SUPPORT POLICY FOR UNMANAGED OPERATORS

The *management state* of an Operator determines whether an Operator is actively managing the resources for its related component in the cluster as designed. If an Operator is set to an *unmanaged* state, it does not respond to changes in configuration nor does it receive updates.

While this can be helpful in non-production clusters or during debugging, Operators in an unmanaged state are unsupported and the cluster administrator assumes full control of the individual component configurations and upgrades.

An Operator can be set to an unmanaged state using the following methods:

- **Individual Operator configuration**
  Individual Operators have a **managementState** parameter in their configuration. This can be accessed in different ways, depending on the Operator. For example, the Red Hat OpenShift Logging Operator accomplishes this by modifying a custom resource (CR) that it manages, while the Cluster Samples Operator uses a cluster-wide configuration resource.

  Changing the **managementState** parameter to **Unmanaged** means that the Operator is not actively managing its resources and will take no action related to the related component. Some Operators might not support this management state as it might damage the cluster and require manual recovery.



WARNING

Changing individual Operators to the **Unmanaged** state renders that particular component and functionality unsupported. Reported issues must be reproduced in **Managed** state for support to proceed.

- **Cluster Version Operator (CVO) overrides**
  The **spec.overrides** parameter can be added to the CVO's configuration to allow administrators to provide a list of overrides to the CVO's behavior for a component. Setting the **spec.overrides[].unmanaged** parameter to **true** for a component blocks cluster upgrades and alerts the administrator after a CVO override has been set:

  > Disabling ownership via cluster version overrides prevents upgrades. Please remove overrides before continuing.

  WARNING

  Setting a CVO override puts the entire cluster in an unsupported state. Reported issues must be reproduced after removing any overrides for support to proceed.

3.4. NEXT STEPS

- Selecting a cluster installation method and preparing it for users

CHAPTER 4. RED HAT OPENSHIFT CLUSTER MANAGER

Red Hat OpenShift Cluster Manager is a managed service where you can install, modify, operate, and upgrade your Red Hat OpenShift clusters. This service allows you to work with all of your organization´s clusters from a single dashboard.

OpenShift Cluster Manager guides you to install OpenShift Container Platform, Red Hat OpenShift Service on AWS (classic architecture), Red Hat OpenShift Service on AWS, and OpenShift Dedicated clusters. It is also responsible for managing both OpenShift Container Platform clusters after self-installation as well as your Red Hat OpenShift Service on AWS (classic architecture) and OpenShift Dedicated clusters.

You can use OpenShift Cluster Manager to do the following actions:

- Create new clusters
- View cluster details and metrics
- Manage your clusters with tasks such as scaling, changing node labels, networking, authentication
- Manage access control
- Monitor clusters
- Schedule upgrades

4.1. ACCESSING RED HAT OPENSHIFT CLUSTER MANAGER

You can access OpenShift Cluster Manager with your configured OpenShift account.

Prerequisites

- You have an account that is part of an OpenShift organization.
- If you are creating a cluster, your organization has a specified quota.

Procedure

- Log in to OpenShift Cluster Manager using your login credentials.

4.2. GENERAL ACTIONS

On the top right of the cluster page, there are some actions that a user can perform on the entire cluster:

- **Open console** launches a web console so that the cluster owner can issue commands to the cluster.
- **Actions** drop-down menu allows the cluster owner to rename the display name of the cluster, change the amount of load balancers and persistent storage on the cluster, if applicable, manually set the node count, and delete the cluster.
- **Refresh** icon forces a refresh of the cluster.

4.3. CLUSTER TABS

Selecting an active, installed cluster shows tabs associated with that cluster. The following tabs display after the cluster's installation completes:

- Overview
- Access control
- Add-ons
- Networking
- Insights Advisor
- Machine pools
- Support
- Settings

4.3.1. Overview tab

The **Overview** tab provides information about how the cluster was configured:

- **Cluster ID** is the unique identification for the created cluster. This ID can be used when issuing commands to the cluster from the command line.
- **Domain prefix** is the prefix that is used throughout the cluster. The default value is the cluster's name.
- **Type** shows the type of cluster, for example ROSA (classic), ROSA with HCP, or Dedicated.
- **Control plane type** is the architecture type of the cluster. The field only displays if the cluster uses a hosted control plane architecture.
- **Region** is the server region.
- **Version** is the OpenShift version that is installed on the cluster. If there is an update available, you can update from this field.
- **Created at** shows the date and time that the cluster was created.
- **Owner** identifies who created the cluster and has owner rights.
- **Delete Protection: <status>** shows whether or not the cluster's delete protection is enabled.
- **Total vCPU** shows the total available virtual CPU for this cluster.
- **Total memory** shows the total available memory for this cluster.
- **Infrastructure AWS account** displays the AWS account that is responsible for cluster creation and maintenance.
- **Nodes** shows the actual and desired nodes on the cluster. These numbers might not match due to cluster scaling.

- **Network** field shows the address and prefixes for network connectivity.
- **OIDC configuration** field shows the Open ID Connect configuration for the cluster.
- **Resource usage** section of the tab displays the resources in use with a graph.
- **Advisor recommendations** section gives insight in relation to security, performance, availability, and stability. This section requires the use of remote health functionality. See *Using Insights to identify issues with the cluster* in the *Additional resources* section.

4.3.2. Access control tab

The **Access control** tab allows the cluster owner to set up an identity provider, grant elevated permissions, and grant roles to other users.

4.3.2.1. Identity providers

You can create your cluster's identity provider in this section. See the *Additional resources* for more information.

4.3.2.2. Cluster roles and acess

You can create a **dedicated-admins** role for {product-short-name} clusters or **cluster-admins** role for OpenShift Container Platform clusters.

Procedure

1. Click the **Add user** button.
2. Enter the ID of the user you want to grant cluster admin access.
3. Select the appropriate group for your user. Either **dedicated-admins** for {product-short-name} clusters, or **cluster-admins** for clusters.

4.3.2.3. OCM roles and access

Prerequisites

- You must be the cluster owner or have the correct permissions to grant roles on the cluster.

Procedure

1. Click the **Grant role** button.
2. Enter the Red Hat account login for the user that you wish to grant a role on the cluster.
3. Select the role from following options:
   - **Cluster editor** allows users or groups to manage or configure the cluster.
   - **Cluster viewer** allows users or groups to view cluster details only.
   - **Identity provider editor** allows users or groups to manage and configure the identity providers.

- **Machine pool editor** allows users or groups to manage and configure the machine pools.

4. Click the **Grant role** button on the dialog box.

4.3.3. Add-ons tab

4.3.4. Insights Advisor tab

The **Insights Advisor** tab uses the Remote Health functionality of the OpenShift Container Platform to identify and mitigate risks to security, performance, availability, and stability. See Using Insights to identify issues with your cluster in the OpenShift Container Platform documentation.

4.3.5. Machine pools tab

The **Machine pools** tab allows the cluster owner to create new machine pools if there is enough available quota, or edit an existing machine pool.

Selecting the  > **Edit** option opens the "Edit machine pool" dialog. In this dialog, you can change the node count per availability zone, edit node labels and taints, and view any associated AWS security groups.

4.3.6. Support tab

In the **Support** tab, you can add notification contacts for individuals that should receive cluster notifications. The username or email address that you provide must relate to a user account in the Red Hat organization where the cluster is deployed.

Also from this tab, you can open a support case to request technical support for your cluster.

4.3.7. Settings tab

The **Settings** tab provides a few options for the cluster owner:

- **Update strategy** allows you to determine if the cluster automatically updates on a certain day of the week at a specified time or if all updates are scheduled manually.
- **Update status** shows the current version and if there are any updates available.

4.4. ADDITIONAL RESOURCES

- For the complete documentation for OpenShift Cluster Manager, see OpenShift Cluster Manager documentation.

CHAPTER 5. ABOUT THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR

One of the challenges of scaling Kubernetes environments is managing the lifecycle of a growing fleet. To meet that challenge, you can use the multicluster engine Operator. The operator delivers full lifecycle capabilities for managed OpenShift Container Platform clusters and partial lifecycle management for other Kubernetes distributions. It is available in two ways:

- As a standalone operator that you install as part of your OpenShift Container Platform or OpenShift Kubernetes Engine subscription
- As part of Red Hat Advanced Cluster Management for Kubernetes

5.1. CLUSTER MANAGEMENT WITH MULTICLUSTER ENGINE ON OPENSHIFT CONTAINER PLATFORM

When you enable multicluster engine on OpenShift Container Platform, you gain the following capabilities:

- Hosted control planes, which is a feature that is based on the HyperShift project. With a centralized hosted control plane, you can operate OpenShift Container Platform clusters in a hyperscale manner.
- Hive, which provisions self-managed OpenShift Container Platform clusters to the hub and completes the initial configurations for those clusters.
- klusterlet agent, which registers managed clusters to the hub.
- Infrastructure Operator, which manages the deployment of the Assisted Service to orchestrate on-premise bare metal and vSphere installations of OpenShift Container Platform, such as single-node OpenShift on bare metal. The Infrastructure Operator includes GitOps Zero Touch Provisioning (ZTP), which fully automates cluster creation on bare metal and vSphere provisioning with GitOps workflows to manage deployments and configuration changes.
- Open cluster management, which provides resources to manage Kubernetes clusters.

The multicluster engine is included with your OpenShift Container Platform support subscription and is delivered separately from the core payload. To start to use multicluster engine, you deploy the OpenShift Container Platform cluster and then install the operator. For more information, see Installing and upgrading multicluster engine operator.

5.2. CLUSTER MANAGEMENT WITH RED HAT ADVANCED CLUSTER MANAGEMENT

If you need cluster management capabilities beyond what OpenShift Container Platform with multicluster engine can provide, consider Red Hat Advanced Cluster Management. The multicluster engine is an integral part of Red Hat Advanced Cluster Management and is enabled by default.

5.3. ADDITIONAL RESOURCES

For the complete documentation for multicluster engine, see Cluster lifecycle with multicluster engine documentation, which is part of the product documentation for Red Hat Advanced Cluster Management.

CHAPTER 6. CONTROL PLANE ARCHITECTURE

The *control plane*, which is composed of control plane machines, manages the OpenShift Container Platform cluster. The control plane machines manage workloads on the compute machines, which are also known as worker machines. The cluster itself manages all upgrades to the machines by the actions of the Cluster Version Operator (CVO), the Machine Config Operator, and a set of individual Operators.

6.1. NODE CONFIGURATION MANAGEMENT WITH MACHINE CONFIG POOLS

Machines that run control plane components or user workloads are divided into groups based on the types of resources they handle. These groups of machines are called machine config pools (MCP). Each MCP manages a set of nodes and its corresponding machine configs. The role of the node determines which MCP it belongs to; the MCP governs nodes based on its assigned node role label. Nodes in an MCP have the same configuration; this means nodes can be scaled up and torn down in response to increased or decreased workloads.

By default, there are two MCPs created by the cluster when it is installed: **master** and **worker**. Each default MCP has a defined configuration applied by the Machine Config Operator (MCO), which is responsible for managing MCPs and facilitating MCP updates.

For worker nodes, you can create additional MCPs, or custom pools, to manage nodes with custom use cases that extend outside of the default node types. Custom MCPs for the control plane nodes are not supported.

Custom pools are pools that inherit their configurations from the worker pool. They use any machine config targeted for the worker pool, but add the ability to deploy changes only targeted at the custom pool. Since a custom pool inherits its configuration from the worker pool, any change to the worker pool is applied to the custom pool as well. Custom pools that do not inherit their configurations from the worker pool are not supported by the MCO.



NOTE

A node can only be included in one MCP. If a node has multiple labels that correspond to several MCPs, like **worker,infra**, it is managed by the infra custom pool, not the worker pool. Custom pools take priority on selecting nodes to manage based on node labels; nodes that do not belong to a custom pool are managed by the worker pool.

It is recommended to have a custom pool for every node role you want to manage in your cluster. For example, if you create infra nodes to handle infra workloads, it is recommended to create a custom infra MCP to group those nodes together. If you apply an **infra** role label to a worker node so it has the **worker,infra** dual label, but do not have a custom infra MCP, the MCO considers it a worker node. If you remove the **worker** label from a node and apply the **infra** label without grouping it in a custom pool, the node is not recognized by the MCO and is unmanaged by the cluster.



IMPORTANT

Any node labeled with the **infra** role that is only running infra workloads is not counted toward the total number of subscriptions. The MCP managing an infra node is mutually exclusive from how the cluster determines subscription charges; tagging a node with the appropriate **infra** role and using taints to prevent user workloads from being scheduled on that node are the only requirements for avoiding subscription charges for infra workloads.

The MCO applies updates for pools independently; for example, if there is an update that affects all pools, nodes from each pool update in parallel with each other. If you add a custom pool, nodes from that pool also attempt to update concurrently with the master and worker nodes.

There might be situations where the configuration on a node does not fully match what the currently-applied machine config specifies. This state is called *configuration drift*. The Machine Config Daemon (MCD) regularly checks the nodes for configuration drift. If the MCD detects configuration drift, the MCO marks the node **degraded** until an administrator corrects the node configuration. A degraded node is online and operational, but, it cannot be updated.

Additional resources

- Understanding configuration drift detection

6.2. MACHINE ROLES IN OPENSHIFT CONTAINER PLATFORM

OpenShift Container Platform assigns hosts different roles. These roles define the function of the machine within the cluster. The cluster contains definitions for the standard **master** and **worker** role types.



NOTE

The cluster also contains the definition for the **bootstrap** role. Because the bootstrap machine is used only during cluster installation, its function is explained in the cluster installation documentation.

6.2.1. Control plane and node host compatibility

The OpenShift Container Platform version must match between control plane host and node host. For example, in a 4.19 cluster, all control plane hosts must be 4.19 and all nodes must be 4.19.

Temporary mismatches during cluster upgrades are acceptable. For example, when upgrading from the previous OpenShift Container Platform version to 4.19, some nodes will upgrade to 4.19 before others. Prolonged skewing of control plane hosts and node hosts might expose older compute machines to bugs and missing features. Users should resolve skewed control plane hosts and node hosts as soon as possible.

The **kubelet** service must not be newer than **kube-apiserver**, and can be up to two minor versions older depending on whether your OpenShift Container Platform version is odd or even. The table below shows the appropriate version compatibility:

OpenShift Container Platform version	Supported **kubelet** skew
Odd OpenShift Container Platform minor versions [1]	Up to one version older
Even OpenShift Container Platform minor versions [2]	Up to two versions older

1. For example, OpenShift Container Platform 4.11, 4.13.
2. For example, OpenShift Container Platform 4.10, 4.12.

6.2.2. Cluster workers

In a Kubernetes cluster, worker nodes run and manage the actual workloads requested by Kubernetes users. The worker nodes advertise their capacity and the scheduler, which is a control plane service, determines on which nodes to start pods and containers. The following important services run on each worker node:

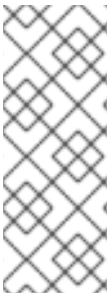
- CRI-O, which is the container engine.
- kubelet, which is the service that accepts and fulfills requests for running and stopping container workloads.
- A service proxy, which manages communication for pods across workers.
- The crun or runC low-level container runtime, which creates and runs containers.



NOTE

For information about how to enable runC instead of the default crun, see the documentation for creating a **ContainerRuntimeConfig** CR.

In OpenShift Container Platform, compute machine sets control the compute machines, which are assigned the **worker** machine role. Machines with the **worker** role drive compute workloads that are governed by a specific machine pool that autoscales them. Because OpenShift Container Platform has the capacity to support multiple machine types, the machines with the **worker** role are classed as *compute* machines. In this release, the terms *worker machine* and *compute machine* are used interchangeably because the only default type of compute machine is the worker machine. In future versions of OpenShift Container Platform, different types of compute machines, such as infrastructure machines, might be used by default.



NOTE

Compute machine sets are groupings of compute machine resources under the **machine-api** namespace. Compute machine sets are configurations that are designed to start new compute machines on a specific cloud provider. Conversely, machine config pools (MCPs) are part of the Machine Config Operator (MCO) namespace. An MCP is used to group machines together so the MCO can manage their configurations and facilitate their upgrades.

6.2.3. Cluster control planes

In a Kubernetes cluster, the *master* nodes run services that are required to control the Kubernetes cluster. In OpenShift Container Platform, the control plane is comprised of control plane machines that have a **master** machine role. They contain more than just the Kubernetes services for managing the OpenShift Container Platform cluster.

For most OpenShift Container Platform clusters, control plane machines are defined by a series of standalone machine API resources. For supported cloud provider and OpenShift Container Platform version combinations, control planes can be managed with control plane machine sets. Extra controls apply to control plane machines to prevent you from deleting all of the control plane machines and making the cluster inoperable. Exactly three control plane nodes must be used for all production deployments.



NOTE

On a bare-metal platform, you can scale a cluster to three, four, or five control plane nodes. For best results, use odd size control plane nodes, such as three or five, on a cluster. A four-node control plane has the same etcd one failure tolerance as three nodes. Only a five-node control plane can tolerate two simultaneous failures.

Adding a fourth control plane node means etcd needs three healthy members instead of two. Until the fourth node is healthy, losing one of the original nodes breaks quorum and takes the cluster down.

Services that fall under the Kubernetes category on the control plane include the Kubernetes API server, etcd, the Kubernetes controller manager, and the Kubernetes scheduler.

Table 6.1. Kubernetes services that run on the control plane

Component	Description
Kubernetes API server	The Kubernetes API server validates and configures the data for pods, services, and replication controllers. It also provides a focal point for the shared state of the cluster.
etcd	etcd stores the persistent control plane state while other components watch etcd for changes to bring themselves into the specified state.
Kubernetes controller manager	The Kubernetes controller manager watches etcd for changes to objects such as replication, namespace, and service account controller objects, and then uses the API to enforce the specified state. Several such processes create a cluster with one active leader at a time.
Kubernetes scheduler	The Kubernetes scheduler watches for newly created pods without an assigned node and selects the best node to host the pod.

There are also OpenShift services that run on the control plane, which include the OpenShift API server, OpenShift controller manager, OpenShift OAuth API server, and OpenShift OAuth server.

Table 6.2. OpenShift services that run on the control plane

Component	Description
OpenShift API server	The OpenShift API server validates and configures the data for OpenShift resources, such as projects, routes, and templates. The OpenShift API server is managed by the OpenShift API Server Operator.
OpenShift controller manager	The OpenShift controller manager watches etcd for changes to OpenShift objects, such as project, route, and template controller objects, and then uses the API to enforce the specified state. The OpenShift controller manager is managed by the OpenShift Controller Manager Operator.

Component	Description
OpenShift OAuth API server	The OpenShift OAuth API server validates and configures the data to authenticate to OpenShift Container Platform, such as users, groups, and OAuth tokens. The OpenShift OAuth API server is managed by the Cluster Authentication Operator.
OpenShift OAuth server	Users request tokens from the OpenShift OAuth server to authenticate themselves to the API. The OpenShift OAuth server is managed by the Cluster Authentication Operator.

Some of these services on the control plane machines run as systemd services, while others run as static pods.

Systemd services are appropriate for services that you need to always come up on that particular system shortly after it starts. For control plane machines, those include sshd, which allows remote login. It also includes services such as:

- The CRI-O container engine (crio), which runs and manages the containers. OpenShift Container Platform 4.19 uses CRI-O instead of the Docker Container Engine.
- Kubelet (kubelet), which accepts requests for managing containers on the machine from control plane services.

CRI-O and Kubelet must run directly on the host as systemd services because they need to be running before you can run other containers.

The **installer-*** and **revision-pruner-*** control plane pods must run with root permissions because they write to the **/etc/kubernetes** directory, which is owned by the root user. These pods are in the following namespaces:

- **openshift-etcd**
- **openshift-kube-apiserver**
- **openshift-kube-controller-manager**
- **openshift-kube-scheduler**

Additional resources

- Hosted control planes overview

6.3. OPERATORS IN OPENSHIFT CONTAINER PLATFORM

Operators are among the most important components of OpenShift Container Platform. They are the preferred method of packaging, deploying, and managing services on the control plane. They can also provide advantages to applications that users run.

Operators integrate with Kubernetes APIs and CLI tools such as **kubectl** and the OpenShift CLI (**oc**). They provide the means of monitoring applications, performing health checks, managing over-the-air (OTA) updates, and ensuring that applications remain in your specified state.

Operators also offer a more granular configuration experience. You configure each component by modifying the API that the Operator exposes instead of modifying a global configuration file.

Because CRI-O and the Kubelet run on every node, almost every other cluster function can be managed on the control plane by using Operators. Components that are added to the control plane by using Operators include critical networking and credential services.

While both follow similar Operator concepts and goals, Operators in OpenShift Container Platform are managed by two different systems, depending on their purpose:

Cluster Operators

Managed by the Cluster Version Operator (CVO) and installed by default to perform cluster functions.

Optional add-on Operators

Managed by Operator Lifecycle Manager (OLM) and can be made accessible for users to run in their applications. Also known as *OLM-based Operators*.

6.3.1. Cluster Operators

In OpenShift Container Platform, all cluster functions are divided into a series of default *cluster Operators*. Cluster Operators manage a particular area of cluster functionality, such as cluster-wide application logging, management of the Kubernetes control plane, or the machine provisioning system.

Cluster Operators are represented by a **ClusterOperator** object, which cluster administrators can view in the OpenShift Container Platform web console from the **Administration → Cluster Settings** page. Each cluster Operator provides a simple API for determining cluster functionality. The Operator hides the details of managing the lifecycle of that component. Operators can manage a single component or tens of components, but the end goal is always to reduce operational burden by automating common actions.

Additional resources

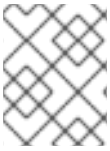
- Cluster Operators reference

6.3.2. Add-on Operators

Operator Lifecycle Manager (OLM) and OperatorHub are default components in OpenShift Container Platform that help manage Kubernetes-native applications as Operators. Together they provide the system for discovering, installing, and managing the optional add-on Operators available on the cluster.

Using OperatorHub in the OpenShift Container Platform web console, cluster administrators and authorized users can select Operators to install from catalogs of Operators. After installing an Operator from OperatorHub, it can be made available globally or in specific namespaces to run in user applications.

Default catalog sources are available that include Red Hat Operators, certified Operators, and community Operators. Cluster administrators can also add their own custom catalog sources, which can contain a custom set of Operators.



NOTE

OLM does not manage the cluster Operators that comprise the OpenShift Container Platform architecture.

Additional resources

- Operator Lifecycle Manager (OLM) concepts and resources
- Understanding OperatorHub.

6.4. OVERVIEW OF ETCD

etcd is a consistent, distributed key-value store that holds small amounts of data that can fit entirely in memory. Although etcd is a core component of many projects, it is the primary data store for Kubernetes, which is the standard system for container orchestration.

6.4.1. Benefits of using etcd

By using etcd, you can benefit in several ways:

- Maintain consistent uptime for your cloud-native applications, and keep them working even if individual servers fail
- Store and replicate all cluster states for Kubernetes
- Distribute configuration data to provide redundancy and resiliency for the configuration of nodes

6.4.2. How etcd works

To ensure a reliable approach to cluster configuration and management, etcd uses the etcd Operator. The Operator simplifies the use of etcd on a Kubernetes container platform like OpenShift Container Platform. With the etcd Operator, you can create or delete etcd members, resize clusters, perform backups, and upgrade etcd.

The etcd Operator observes, analyzes, and acts:

1. It observes the cluster state by using the Kubernetes API.
2. It analyzes differences between the current state and the state that you want.
3. It fixes the differences through the etcd cluster management APIs, the Kubernetes API, or both.

etcd holds the cluster state, which is constantly updated. This state is continuously persisted, which leads to a high number of small changes at high frequency. As a result, it is critical to back the etcd cluster member with fast, low-latency I/O. For more information about best practices for etcd, see "Recommended etcd practices".

Additional resources

- Recommended etcd practices
- Backing up etcd

CHAPTER 7. UNDERSTANDING OPENSHIFT CONTAINER PLATFORM DEVELOPMENT

To fully leverage the capability of containers when developing and running enterprise-quality applications, ensure your environment is supported by tools that allow containers to be:

- Created as discrete microservices that can be connected to other containerized, and non-containerized, services. For example, you might want to join your application with a database or attach a monitoring application to it.
- Resilient, so if a server crashes or needs to go down for maintenance or to be decommissioned, containers can start on another machine.
- Automated to pick up code changes automatically and then start and deploy new versions of themselves.
- Scaled up, or replicated, to have more instances serving clients as demand increases and then spun down to fewer instances as demand declines.
- Run in different ways, depending on the type of application. For example, one application might run once a month to produce a report and then exit. Another application might need to run constantly and be highly available to clients.
- Managed so you can watch the state of your application and react when something goes wrong.

Containers' widespread acceptance, and the resulting requirements for tools and methods to make them enterprise-ready, resulted in many options for them.

The rest of this section explains options for assets you can create when you build and deploy containerized Kubernetes applications in OpenShift Container Platform. It also describes which approaches you might use for different kinds of applications and development requirements.

7.1. ABOUT DEVELOPING CONTAINERIZED APPLICATIONS

You can approach application development with containers in many ways, and different approaches might be more appropriate for different situations. To illustrate some of this variety, the series of approaches that is presented starts with developing a single container and ultimately deploys that container as a mission-critical application for a large enterprise. These approaches show different tools, formats, and methods that you can employ with containerized application development. This topic describes:

- Building a simple container and storing it in a registry
- Creating a Kubernetes manifest and saving it to a Git repository
- Making an Operator to share your application with others

7.2. BUILDING A SIMPLE CONTAINER

You have an idea for an application and you want to containerize it.

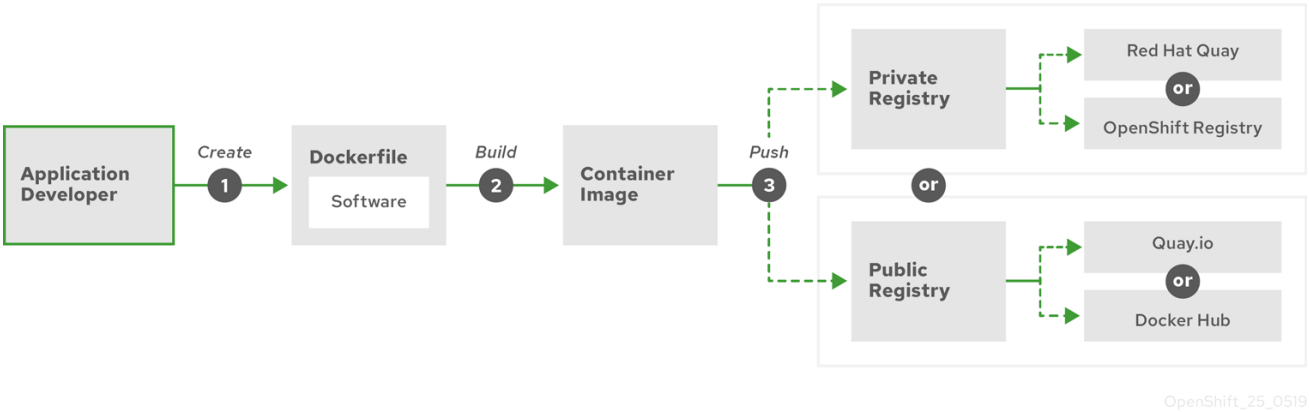
First you require a tool for building a container, like buildah or docker, and a file that describes what goes in your container, which is typically a Dockerfile.

Next, you require a location to push the resulting container image so you can pull it to run anywhere you want it to run. This location is a container registry.

Some examples of each of these components are installed by default on most Linux operating systems, except for the Dockerfile, which you provide yourself.

The following diagram displays the process of building and pushing an image:

Figure 7.1. Create a simple containerized application and push it to a registry



If you use a computer that runs Red Hat Enterprise Linux (RHEL) as the operating system, the process of creating a containerized application requires the following steps:

1. Install container build tools: RHEL contains a set of tools that includes podman, buildah, and skopeo that you use to build and manage containers.

2. Create a Dockerfile to combine base image and software: Information about building your container goes into a file that is named **Dockerfile**. In that file, you identify the base image you build from, the software packages you install, and the software you copy into the container. You also identify parameter values like network ports that you expose outside the container and volumes that you mount inside the container. Put your Dockerfile and the software you want to containerize in a directory on your RHEL system.

3. Run buildah or docker build: Run the **buildah build-using-dockerfile** or the **docker build** command to pull your chosen base image to the local system and create a container image that is stored locally. You can also build container images without a Dockerfile by using buildah.

4. Tag and push to a registry: Add a tag to your new container image that identifies the location of the registry in which you want to store and share your container. Then push that image to the registry by running the **podman push** or **docker push** command.

5. Pull and run the image: From any system that has a container client tool, such as podman or docker, run a command that identifies your new image. For example, run the **podman run <image_name>** or **docker run <image_name>** command. Here **<image_name>** is the name of your new container image, which resembles **quay.io/myrepo/myapp:latest**. The registry might require credentials to push and pull images.

For more details on the process of building container images, pushing them to registries, and running them, see Custom image builds with Buildah .

7.2.1. Container build tool options

Building and managing containers with buildah, podman, and skopeo results in industry standard container images that include features specifically tuned for deploying containers in OpenShift Container Platform or other Kubernetes environments. These tools are daemonless and can run without root privileges, requiring less overhead to run them.



IMPORTANT

Support for Docker Container Engine as a container runtime is deprecated in Kubernetes 1.20 and will be removed in a future release. However, Docker-produced images will continue to work in your cluster with all runtimes, including CRI-O. For more information, see the Kubernetes blog announcement.

When you ultimately run your containers in OpenShift Container Platform, you use the CRI-O container engine. CRI-O runs on every worker and control plane machine in an OpenShift Container Platform cluster, but CRI-O is not yet supported as a standalone runtime outside of OpenShift Container Platform.

7.2.2. Base image options

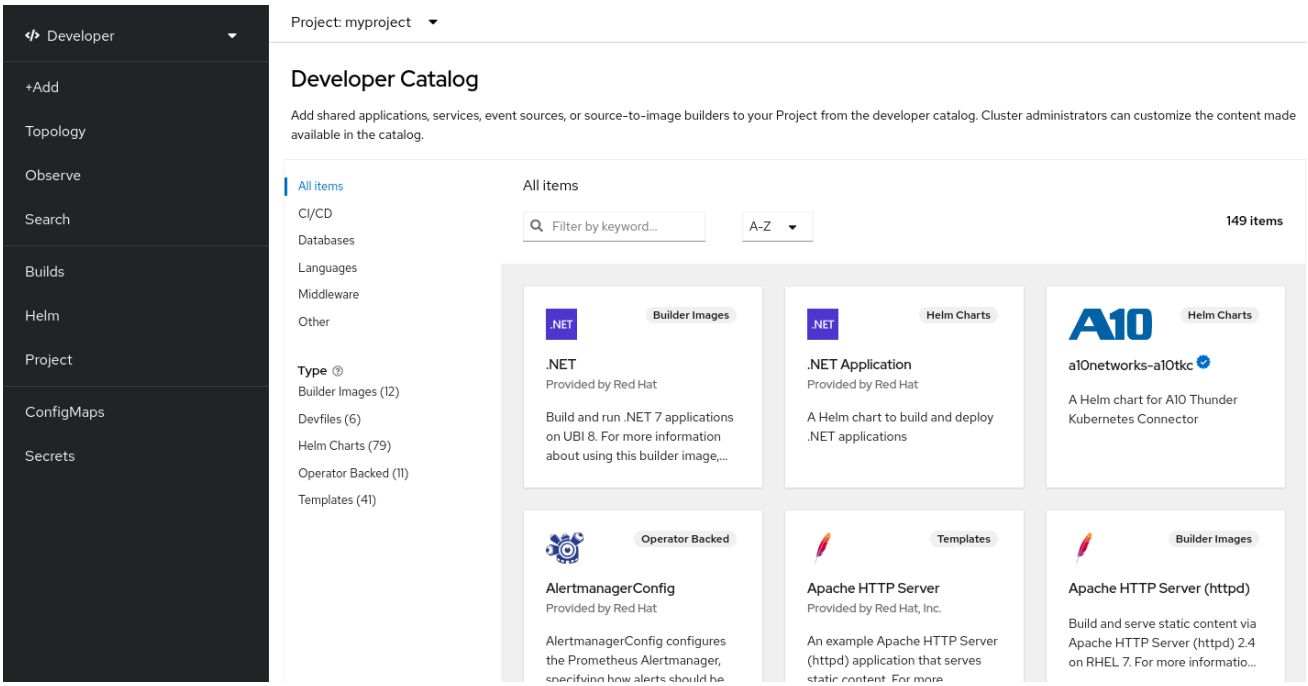
The base image you choose to build your application on contains a set of software that resembles a Linux system to your application. When you build your own image, your software is placed into that file system and sees that file system as though it were looking at its operating system. Choosing this base image has major impact on how secure, efficient and upgradeable your container is in the future.

Red Hat provides a new set of base images referred to as Red Hat Universal Base Images (UBI). These images are based on Red Hat Enterprise Linux and are similar to base images that Red Hat has offered in the past, with one major difference: they are freely redistributable without a Red Hat subscription. As a result, you can build your application on UBI images without having to worry about how they are shared or the need to create different images for different environments.

These UBI images have standard, init, and minimal versions. You can also use the Red Hat Software Collections images as a foundation for applications that rely on specific runtime environments such as Node.js, Perl, or Python. Special versions of some of these runtime base images are referred to as Source-to-Image (S2I) images. With S2I images, you can insert your code into a base image environment that is ready to run that code.

S2I images are available for you to use directly from the OpenShift Container Platform web UI. In the Developer perspective, navigate to the **+Add** view and in the **Developer Catalog** tile, view all of the available services in the Developer Catalog.

Figure 7.2. Choose S2I base images for apps that need specific runtimes



7.2.3. Registry options

Container registries are where you store container images so you can share them with others and make them available to the platform where they ultimately run. You can select large, public container registries that offer free accounts or a premium version that offer more storage and special features. You can also install your own registry that can be exclusive to your organization or selectively shared with others.

To get Red Hat images and certified partner images, you can draw from the Red Hat Registry. The Red Hat Registry is represented by two locations: **registry.access.redhat.com**, which is unauthenticated and deprecated, and **registry.redhat.io**, which requires authentication. You can learn about the Red Hat and partner images in the Red Hat Registry from the Container images section of the Red Hat Ecosystem Catalog. Besides listing Red Hat container images, it also shows extensive information about the contents and quality of those images, including health scores that are based on applied security updates.

Large, public registries include Docker Hub and Quay.io. The Quay.io registry is owned and managed by Red Hat. Many of the components used in OpenShift Container Platform are stored in Quay.io, including container images and the Operators that are used to deploy OpenShift Container Platform itself. Quay.io also offers the means of storing other types of content, including Helm charts.

If you want your own, private container registry, OpenShift Container Platform itself includes a private container registry that is installed with OpenShift Container Platform and runs on its cluster. Red Hat also offers a private version of the Quay.io registry called Red Hat Quay. Red Hat Quay includes geo replication, Git build triggers, Clair image scanning, and many other features.

All of the registries mentioned here can require credentials to download images from those registries. Some of those credentials are presented on a cluster-wide basis from OpenShift Container Platform, while other credentials can be assigned to individuals.

7.3. CREATING A KUBERNETES MANIFEST FOR OPENSHIFT CONTAINER PLATFORM

While the container image is the basic building block for a containerized application, more information is required to manage and deploy that application in a Kubernetes environment such as OpenShift Container Platform. The typical next steps after you create an image are to:

- Understand the different resources you work with in Kubernetes manifests
- Make some decisions about what kind of an application you are running
- Gather supporting components
- Create a manifest and store that manifest in a Git repository so you can store it in a source versioning system, audit it, track it, promote and deploy it to the next environment, roll it back to earlier versions, if necessary, and share it with others

7.3.1. About Kubernetes pods and services

While the container image is the basic unit with docker, the basic units that Kubernetes works with are called pods. Pods represent the next step in building out an application. A pod can contain one or more than one container. The key is that the pod is the single unit that you deploy, scale, and manage.

Scalability and namespaces are probably the main items to consider when determining what goes in a pod. For ease of deployment, you might want to deploy a container in a pod and include its own logging and monitoring container in the pod. Later, when you run the pod and need to scale up an additional instance, those other containers are scaled up with it. For namespaces, containers in a pod share the same network interfaces, shared storage volumes, and resource limitations, such as memory and CPU, which makes it easier to manage the contents of the pod as a single unit. Containers in a pod can also communicate with each other by using standard inter-process communications, such as System V semaphores or POSIX shared memory.

While individual pods represent a scalable unit in Kubernetes, a service provides a means of grouping together a set of pods to create a complete, stable application that can complete tasks such as load balancing. A service is also more permanent than a pod because the service remains available from the same IP address until you delete it. When the service is in use, it is requested by name and the OpenShift Container Platform cluster resolves that name into the IP addresses and ports where you can reach the pods that compose the service.

By their nature, containerized applications are separated from the operating systems where they run and, by extension, their users. Part of your Kubernetes manifest describes how to expose the application to internal and external networks by defining network policies that allow fine-grained control over communication with your containerized applications. To connect incoming requests for HTTP, HTTPS, and other services from outside your cluster to services inside your cluster, you can use an **Ingress** resource.

If your container requires on-disk storage instead of database storage, which might be provided through a service, you can add volumes to your manifests to make that storage available to your pods. You can configure the manifests to create persistent volumes (PVs) or dynamically create volumes that are added to your **Pod** definitions.

After you define a group of pods that compose your application, you can define those pods in **Deployment** and **DeploymentConfig** objects.

7.3.2. Application types

Next, consider how your application type influences how to run it.

Kubernetes defines different types of workloads that are appropriate for different kinds of applications. To determine the appropriate workload for your application, consider if the application is:

- Meant to run to completion and be done. An example is an application that starts up to produce a report and exits when the report is complete. The application might not run again then for a month. Suitable OpenShift Container Platform objects for these types of applications include **Job** and **CronJob** objects.

- Expected to run continuously. For long-running applications, you can write a deployment.

- Required to be highly available. If your application requires high availability, then you want to size your deployment to have more than one instance. A **Deployment** or **DeploymentConfig** object can incorporate a replica set for that type of application. With replica sets, pods run across multiple nodes to make sure the application is always available, even if a worker goes down.

- Need to run on every node. Some types of Kubernetes applications are intended to run in the cluster itself on every master or worker node. DNS and monitoring applications are examples of applications that need to run continuously on every node. You can run this type of application as a daemon set. You can also run a daemon set on a subset of nodes, based on node labels.

- Require life-cycle management. When you want to hand off your application so that others can use it, consider creating an Operator. Operators let you build in intelligence, so it can handle things like backups and upgrades automatically. Coupled with the Operator Lifecycle Manager (OLM), cluster managers can expose Operators to selected namespaces so that users in the cluster can run them.

- Have identity or numbering requirements. An application might have identity requirements or numbering requirements. For example, you might be required to run exactly three instances of the application and to name the instances **0**, **1**, and **2**. A stateful set is suitable for this application. Stateful sets are most useful for applications that require independent storage, such as databases and zookeeper clusters.

7.3.3. Available supporting components

The application you write might need supporting components, like a database or a logging component. To fulfill that need, you might be able to obtain the required component from the following Catalogs that are available in the OpenShift Container Platform web console:

- OperatorHub, which is available in each OpenShift Container Platform 4.19 cluster. The OperatorHub makes Operators available from Red Hat, certified Red Hat partners, and community members to the cluster operator. The cluster operator can make those Operators available in all or selected namespaces in the cluster, so developers can launch them and configure them with their applications.

- Templates, which are useful for a one-off type of application, where the lifecycle of a component is not important after it is installed. A template provides an easy way to get started developing a Kubernetes application with minimal overhead. A template can be a list of resource definitions, which could be **Deployment**, **Service**, **Route**, or other objects. If you want to change names or resources, you can set these values as parameters in the template.

You can configure the supporting Operators and templates to the specific needs of your development team and then make them available in the namespaces in which your developers work. Many people add shared templates to the **openshift** namespace because it is accessible from all other namespaces.

7.3.4. Applying the manifest

Kubernetes manifests let you create a more complete picture of the components that make up your Kubernetes applications. You write these manifests as YAML files and deploy them by applying them to the cluster, for example, by running the **oc apply** command.

7.3.5. Next steps

At this point, consider ways to automate your container development process. Ideally, you have some sort of CI pipeline that builds the images and pushes them to a registry. In particular, a GitOps pipeline integrates your container development with the Git repositories that you use to store the software that is required to build your applications.

The workflow to this point might look like:

- Day 1: You write some YAML. You then run the **oc apply** command to apply that YAML to the cluster and test that it works.
- Day 2: You put your YAML container configuration file into your own Git repository. From there, people who want to install that app, or help you improve it, can pull down the YAML and apply it to their cluster to run the app.
- Day 3: Consider writing an Operator for your application.

7.4. DEVELOP FOR OPERATORS

Packaging and deploying your application as an Operator might be preferred if you make your application available for others to run. As noted earlier, Operators add a lifecycle component to your application that acknowledges that the job of running an application is not complete as soon as it is installed.

When you create an application as an Operator, you can build in your own knowledge of how to run and maintain the application. You can build in features for upgrading the application, backing it up, scaling it, or keeping track of its state. If you configure the application correctly, maintenance tasks, like updating the Operator, can happen automatically and invisibly to the Operator's users.

An example of a useful Operator is one that is set up to automatically back up data at particular times. Having an Operator manage an application's backup at set times can save a system administrator from remembering to do it.

Any application maintenance that has traditionally been completed manually, like backing up data or rotating certificates, can be completed automatically with an Operator.

CHAPTER 8. RED HAT ENTERPRISE LINUX COREOS (RHCOS)

8.1. ABOUT RHCOS

Red Hat Enterprise Linux CoreOS (RHCOS) represents the next generation of single-purpose container operating system technology by providing the quality standards of Red Hat Enterprise Linux (RHEL) with automated, remote upgrade features.

RHCOS is supported only as a component of OpenShift Container Platform 4.19 for all OpenShift Container Platform machines. RHCOS is the only supported operating system for all node types in OpenShift Container Platform. RHCOS is deployed in OpenShift Container Platform 4.19 in two general ways:

- If you install your cluster on infrastructure that the installation program provisions, RHCOS images are downloaded to the target platform during installation. Suitable Ignition config files, which control the RHCOS configuration, are also downloaded and used to deploy the machines.

- If you install your cluster on infrastructure that you manage, you must follow the installation documentation to obtain the RHCOS images, generate Ignition config files, and use the Ignition config files to provision your machines.

8.1.1. Key RHCOS features

The following list describes key features of the RHCOS operating system:

- **Based on RHEL**: The underlying operating system consists primarily of RHEL components. The same quality, security, and control measures that support RHEL also support RHCOS. For example, RHCOS software is in RPM packages, and each RHCOS system starts up with a RHEL kernel and a set of services that are managed by the systemd init system.

- **Controlled immutability**: Although it contains RHEL components, RHCOS is designed to be managed more tightly than a default RHEL installation. Management is performed remotely from the OpenShift Container Platform cluster. When you set up your RHCOS machines, you can modify only a few system settings. This controlled immutability allows OpenShift Container Platform to store the latest state of RHCOS systems in the cluster so it is always able to create additional machines and perform updates based on the latest RHCOS configurations.

- **CRI-O container runtime**: Although RHCOS contains features for running the OCI- and libcontainer-formatted containers that Docker requires, it incorporates the CRI-O container engine instead of the Docker container engine. By focusing on features needed by Kubernetes platforms, such as OpenShift Container Platform, CRI-O can offer specific compatibility with different Kubernetes versions. CRI-O also offers a smaller footprint and reduced attack surface than is possible with container engines that offer a larger feature set. At the moment, CRI-O is the only engine available within OpenShift Container Platform clusters.
  CRI-O can use either the crun or runC container runtime to start and manage containers. crun is the default. For information about how to enable runC, see the documentation for creating a **ContainerRuntimeConfig** CR.

- **Set of container tools**: For tasks such as building, copying, and otherwise managing containers, RHCOS replaces the Docker CLI tool with a compatible set of container tools. The podman CLI tool supports many container runtime features, such as running, starting, stopping, listing, and removing containers and container images. The skopeo CLI tool can copy, authenticate, and sign images. You can use the **crictl** CLI tool to work with containers and pods from the CRI-O container engine. While direct use of these tools in RHCOS is discouraged, you can use them for debugging purposes.

- **rpm-ostree upgrades**: RHCOS features transactional upgrades using the **rpm-ostree** system. Updates are delivered by means of container images and are part of the OpenShift Container Platform update process. When deployed, the container image is pulled, extracted, and written to disk, then the bootloader is modified to boot into the new version. The machine will reboot into the update in a rolling manner to ensure cluster capacity is minimally impacted.

- **bootupd firmware and bootloader updater**: Package managers and hybrid systems such as **rpm-ostree** do not update the firmware or the bootloader. With **bootupd**, RHCOS users have access to a cross-distribution, system-agnostic update tool that manages firmware and boot updates in UEFI and legacy BIOS boot modes that run on modern architectures, such as x86_64, ppc64le, and aarch64.
  For information about how to install **bootupd**, see the documentation for *Updating the bootloader using bootupd*.

- **Updated through the Machine Config Operator**: In OpenShift Container Platform, the Machine Config Operator handles operating system upgrades. Instead of upgrading individual packages, as is done with **yum** upgrades, **rpm-ostree** delivers upgrades of the OS as an atomic unit. The new OS deployment is staged during upgrades and goes into effect on the next reboot. If something goes wrong with the upgrade, a single rollback and reboot returns the system to the previous state. RHCOS upgrades in OpenShift Container Platform are performed during cluster updates.

For RHCOS systems, the layout of the **rpm-ostree** file system has the following characteristics:

- /**usr** is where the operating system binaries and libraries are stored and is read-only. We do not support altering this.

- /**etc**, /**boot**, /**var** are writable on the system but only intended to be altered by the Machine Config Operator.

- /**var**/**lib**/**containers** is the graph storage location for storing container images.

8.1.2. Choosing how to configure RHCOS

RHCOS is designed to deploy on an OpenShift Container Platform cluster with a minimal amount of user configuration. In its most basic form, this consists of:

- Starting with a provisioned infrastructure, such as on AWS, or provisioning the infrastructure yourself.

- Supplying a few pieces of information, such as credentials and cluster name, in an **install-config.yaml** file when running **openshift-install**.

Because RHCOS systems in OpenShift Container Platform are designed to be fully managed from the OpenShift Container Platform cluster after that, directly changing an RHCOS machine is discouraged. Although limited direct access to RHCOS machines cluster can be accomplished for debugging purposes, you should not directly configure RHCOS systems. Instead, if you need to add or change features on your OpenShift Container Platform nodes, consider making changes in the following ways:

- **Kubernetes workload objects, such as DaemonSet and Deployment**: If you need to add services or other user-level features to your cluster, consider adding them as Kubernetes workload objects. Keeping those features outside of specific node configurations is the best way to reduce the risk of breaking the cluster on subsequent upgrades.

- **Day-2 customizations**: If possible, bring up a cluster without making any customizations to cluster nodes and make necessary node changes after the cluster is up. Those changes are

easier to track later and less likely to break updates. Creating machine configs or modifying Operator custom resources are ways of making these customizations.

- **Day-1 customizations**: For customizations that you must implement when the cluster first comes up, there are ways of modifying your cluster so changes are implemented on first boot. Day-1 customizations can be done through Ignition configs and manifest files during **openshift-install** or by adding boot options during ISO installs provisioned by the user.

Here are examples of customizations you could do on day 1:

- **Kernel arguments**: If particular kernel features or tuning is needed on nodes when the cluster first boots.
- **Disk encryption**: If your security needs require that the root file system on the nodes are encrypted, such as with FIPS support.
- **Kernel modules**: If a particular hardware device, such as a network card or video card, does not have a usable module available by default in the Linux kernel.
- **Chronyd**: If you want to provide specific clock settings to your nodes, such as the location of time servers.

To accomplish these tasks, you can augment the **openshift-install** process to include additional objects such as **MachineConfig** objects. Those procedures that result in creating machine configs can be passed to the Machine Config Operator after the cluster is up.



NOTE

- The Ignition config files that the installation program generates contain certificates that expire after 24 hours, which are then renewed at that time. If the cluster is shut down before renewing the certificates and the cluster is later restarted after the 24 hours have elapsed, the cluster automatically recovers the expired certificates. The exception is that you must manually approve the pending **node-bootstrapper** certificate signing requests (CSRs) to recover kubelet certificates. See the documentation for *Recovering from expired control plane certificates* for more information.
- It is recommended that you use Ignition config files within 12 hours after they are generated because the 24-hour certificate rotates from 16 to 22 hours after the cluster is installed. By using the Ignition config files within 12 hours, you can avoid installation failure if the certificate update runs during installation.

8.1.3. Choosing how to deploy RHCOS

Differences between RHCOS installations for OpenShift Container Platform are based on whether you are deploying on an infrastructure provisioned by the installer or by the user:

- **Installer-provisioned**: Some cloud environments offer preconfigured infrastructures that allow you to bring up an OpenShift Container Platform cluster with minimal configuration. For these types of installations, you can supply Ignition configs that place content on each node so it is there when the cluster first boots.
- **User-provisioned**: If you are provisioning your own infrastructure, you have more flexibility in how you add content to a RHCOS node. For example, you could add kernel arguments when you boot the RHCOS ISO installer to install each system. However, in most cases where

configuration is required on the operating system itself, it is best to provide that configuration through an Ignition config.

The Ignition facility runs only when the RHCOS system is first set up. After that, Ignition configs can be supplied later using the machine config.

8.1.4. About Ignition

Ignition is the utility that is used by RHCOS to manipulate disks during initial configuration. It completes common disk tasks, including partitioning disks, formatting partitions, writing files, and configuring users. On first boot, Ignition reads its configuration from the installation media or the location that you specify and applies the configuration to the machines.

Whether you are installing your cluster or adding machines to it, Ignition always performs the initial configuration of the OpenShift Container Platform cluster machines. Most of the actual system setup happens on each machine itself. For each machine, Ignition takes the RHCOS image and boots the RHCOS kernel. Options on the kernel command line identify the type of deployment and the location of the Ignition-enabled initial RAM disk (initramfs).

8.1.4.1. How Ignition works

To create machines by using Ignition, you need Ignition config files. The OpenShift Container Platform installation program creates the Ignition config files that you need to deploy your cluster. These files are based on the information that you provide to the installation program directly or through an **install-config.yaml** file.

The way that Ignition configures machines is similar to how tools like cloud-init or Linux Anaconda kickstart configure systems, but with some important differences:

- Ignition runs from an initial RAM disk that is separate from the system you are installing to. Because of that, Ignition can repartition disks, set up file systems, and perform other changes to the machine's permanent file system. In contrast, cloud-init runs as part of a machine init system when the system boots, so making foundational changes to things like disk partitions cannot be done as easily. With cloud-init, it is also difficult to reconfigure the boot process while you are in the middle of the node boot process.

- Ignition is meant to initialize systems, not change existing systems. After a machine initializes and the kernel is running from the installed system, the Machine Config Operator from the OpenShift Container Platform cluster completes all future machine configuration.

- Instead of completing a defined set of actions, Ignition implements a declarative configuration. It checks that all partitions, files, services, and other items are in place before the new machine starts. It then makes the changes, like copying files to disk that are necessary for the new machine to meet the specified configuration.

- After Ignition finishes configuring a machine, the kernel keeps running but discards the initial RAM disk and pivots to the installed system on disk. All of the new system services and other features start without requiring a system reboot.

- Because Ignition confirms that all new machines meet the declared configuration, you cannot have a partially configured machine. If a machine setup fails, the initialization process does not finish, and Ignition does not start the new machine. Your cluster will never contain partially configured machines. If Ignition cannot complete, the machine is not added to the cluster. You must add a new machine instead. This behavior prevents the difficult case of debugging a machine when the results of a failed configuration task are not known until something that depended on it fails at a later date.

- If there is a problem with an Ignition config that causes the setup of a machine to fail, Ignition will not try to use the same config to set up another machine. For example, a failure could result from an Ignition config made up of a parent and child config that both want to create the same file. A failure in such a case would prevent that Ignition config from being used again to set up an other machines until the problem is resolved.

- If you have multiple Ignition config files, you get a union of that set of configs. Because Ignition is declarative, conflicts between the configs could cause Ignition to fail to set up the machine. The order of information in those files does not matter. Ignition will sort and implement each setting in ways that make the most sense. For example, if a file needs a directory several levels deep, if another file needs a directory along that path, the later file is created first. Ignition sorts and creates all files, directories, and links by depth.

- Because Ignition can start with a completely empty hard disk, it can do something cloud-init cannot do: set up systems on bare metal from scratch using features such as PXE boot. In the bare metal case, the Ignition config is injected into the boot partition so that Ignition can find it and configure the system correctly.

8.1.4.2. The Ignition sequence

The Ignition process for an RHCOS machine in an OpenShift Container Platform cluster involves the following steps:

- The machine gets its Ignition config file. Control plane machines get their Ignition config files from the bootstrap machine, and worker machines get Ignition config files from a control plane machine.

- Ignition creates disk partitions, file systems, directories, and links on the machine. It supports RAID arrays but does not support LVM volumes.

- Ignition mounts the root of the permanent file system to the **/sysroot** directory in the initramfs and starts working in that **/sysroot** directory.

- Ignition configures all defined file systems and sets them up to mount appropriately at runtime.

- Ignition runs **systemd** temporary files to populate required files in the **/var** directory.

- Ignition runs the Ignition config files to set up users, systemd unit files, and other configuration files.

- Ignition unmounts all components in the permanent system that were mounted in the initramfs.

- Ignition starts up the init process of the new machine, which in turn starts up all other services on the machine that run during system boot.

At the end of this process, the machine is ready to join the cluster and does not require a reboot.

8.2. VIEWING IGNITION CONFIGURATION FILES

To see the Ignition config file used to deploy the bootstrap machine, run the following command:

```
$ openshift-install create ignition-configs --dir $HOME/testconfig
```

After you answer a few questions, the **bootstrap.ign**, **master.ign**, and **worker.ign** files appear in the directory you entered.

To see the contents of the **bootstrap.ign** file, pipe it through the **jq** filter. Here's a snippet from that file:

```
$ cat $HOME/testconfig/bootstrap.ign | jq
{
  "ignition": {
    "version": "3.2.0"
  },
  "passwd": {
    "users": [
      {
        "name": "core",
        "sshAuthorizedKeys": [
          "ssh-rsa AAAAB3NzaC1yc...."
        ]
      }
    ]
  },
  "storage": {
    "files": [
      {
        "overwrite": false,
        "path": "/etc/motd",
        "user": {
          "name": "root"
        },
        "append": [
          {
            "source": "data:text/plain;charset=utf-
8; base64,VGhpcyBpcyB0aGUgYm9vdHN0cmFwIG5vZGU7IGl0IHdpbGwgYmUgZGVzdHJveWVkIHdo
ZW4gdGhlIG1hc3RlciBpcyBmdWxseSB1cC4KClRoZSBwcmltYXJ5IHNlcnZpY2VzIGFyZSByZWxlYXNlLWlt
WltYWdlLnNlcnZpY2UgZm9sbG93ZWQgYnkgYm9vdGt1YmUuc2VydmljZS4gVG8gd2F0Y2ggdGhlaXI
gc3RhdHVzLCBydW4gZS5nLgoKICBqb3VybmFsY3RsIC1iIC1mIC11IHJlbGVhc2UtaW1hZ2Uuc2Vydm
ljZSAtdSBib290a3ViZS5zZXJ2aWNlCg=="
          }
        ],
        "mode": 420
      },
...
```

To decode the contents of a file listed in the **bootstrap.ign** file, pipe the base64-encoded data string representing the contents of that file to the **base64 -d** command. Here's an example using the contents of the /**etc**/**motd** file added to the bootstrap machine from the output shown above:

```
$ echo
VGhpcyBpcyB0aGUgYm9vdHN0cmFwIG5vZGU7IGl0IHdpbGwgYmUgZGVzdHJveWVkIHdoZW4gdG
hlIG1hc3RlciBpcyBmdWxseSB1cC4KClRoZSBwcmltYXJ5IHNlcnZpY2VzIGFyZSByZWxlYXNlLWlmYWdl
LnNlcnZpY2UgZm9sbG93ZWQgYnkgYm9vdGt1YmUuc2VydmljZS4gVG8gd2F0Y2ggdGhlaXIgc3Rhd
HVzLCBydW4gZS5nLgoKICBqb3VybmFsY3RsIC1iIC1mIC11IHJlbGVhc2UtaW1hZ2Uuc2Vydm
ljZSAtdSBib290a3ViZS5zZXJ2aWNlCg== | base64 --decode
```

Example output

```
This is the bootstrap node; it will be destroyed when the master is fully up.

The primary services are release-image.service followed by bootkube.service. To watch their status,
```

```
run e.g.

  journalctl -b -f -u release-image.service -u bootkube.service
```

Repeat those commands on the **master.ign** and **worker.ign** files to see the source of Ignition config files for each of those machine types. You should see a line like the following for the **worker.ign**, identifying how it gets its Ignition config from the bootstrap machine:

```
"source": "https://api.myign.develcluster.example.com:22623/config/worker",
```

Here are a few things you can learn from the **bootstrap.ign** file:

- Format: The format of the file is defined in the Ignition config spec. Files of the same format are used later by the MCO to merge changes into a machine's configuration.
- Contents: Because the bootstrap machine serves the Ignition configs for other machines, both master and worker machine Ignition config information is stored in the **bootstrap.ign**, along with the bootstrap machine's configuration.
- Size: The file is more than 1300 lines long, with path to various types of resources.
- The content of each file that will be copied to the machine is actually encoded into data URLs, which tends to make the content a bit clumsy to read. (Use the **jq** and **base64** commands shown previously to make the content more readable.)
- Configuration: The different sections of the Ignition config file are generally meant to contain files that are just dropped into a machine's file system, rather than commands to modify existing files. For example, instead of having a section on NFS that configures that service, you would just add an NFS configuration file, which would then be started by the init process when the system comes up.
- users: A user named **core** is created, with your SSH key assigned to that user. This allows you to log in to the cluster with that user name and your credentials.
- storage: The storage section identifies files that are added to each machine. A few notable files include **/root/.docker/config.json** (which provides credentials your cluster needs to pull from container image registries) and a bunch of manifest files in **/opt/openshift/manifests** that are used to configure your cluster.
- systemd: The **systemd** section holds content used to create **systemd** unit files. Those files are used to start up services at boot time, as well as manage those services on running systems.
- Primitives: Ignition also exposes low-level primitives that other tools can build on.

8.3. CHANGING IGNITION CONFIGS AFTER INSTALLATION

Machine config pools manage a cluster of nodes and their corresponding machine configs. Machine configs contain configuration information for a cluster. To list all machine config pools that are known:

```
$ oc get machineconfigpools
```

Example output

```
NAME   CONFIG                                         UPDATED UPDATING DEGRADED
master master-1638c1aea398413bb918e76632f20799 False   False    False
worker worker-2feef4f8288936489a5a832ca8efe953 False   False    False
```

To list all machine configs:

```
$ oc get machineconfig
```

Example output

```
NAME                                               GENERATEDBYCONTROLLER   IGNITIONVERSION   CREATED
OSIMAGEURL

00-master                                          4.0.0-0.150.0.0-dirty   3.5.0             16m
00-master-ssh                                        4.0.0-0.150.0.0-dirty                   16m
00-worker                                          4.0.0-0.150.0.0-dirty   3.5.0             16m
00-worker-ssh                                        4.0.0-0.150.0.0-dirty                   16m
01-master-kubelet                                    4.0.0-0.150.0.0-dirty   3.5.0             16m
01-worker-kubelet                                    4.0.0-0.150.0.0-dirty   3.5.0             16m
master-1638c1aea398413bb918e76632f20799   4.0.0-0.150.0.0-dirty   3.5.0             16m
worker-2feef4f8288936489a5a832ca8efe953   4.0.0-0.150.0.0-dirty   3.5.0             16m
```

The Machine Config Operator acts somewhat differently than Ignition when it comes to applying these machine configs. The machine configs are read in order (from 00* to 99*). Labels inside the machine configs identify the type of node each is for (master or worker). If the same file appears in multiple machine config files, the last one wins. So, for example, any file that appears in a 99* file would replace the same file that appeared in a 00* file. The input **MachineConfig** objects are unioned into a "rendered" **MachineConfig** object, which will be used as a target by the operator and is the value you can see in the machine config pool.

To see what files are being managed from a machine config, look for "Path:" inside a particular **MachineConfig** object. For example:

```
$ oc describe machineconfigs 01-worker-container-runtime | grep Path:
```

Example output

```
      Path:         /etc/containers/registries.conf
      Path:         /etc/containers/storage.conf
      Path:         /etc/crio/crio.conf
```

Be sure to give the machine config file a later name (such as 10-worker-container-runtime). Keep in mind that the content of each file is in URL-style data. Then apply the new machine config to the cluster.

CHAPTER 9. ADMISSION PLUGINS

Admission plugins are used to help regulate how OpenShift Container Platform functions.

9.1. ABOUT ADMISSION PLUGINS

Admission plugins intercept requests to the master API to validate resource requests. After a request is authenticated and authorized, the admission plugins ensure that any associated policies are followed. For example, they are commonly used to enforce security policy, resource limitations or configuration requirements.

Admission plugins run in sequence as an admission chain. If any admission plugin in the sequence rejects a request, the whole chain is aborted and an error is returned.

OpenShift Container Platform has a default set of admission plugins enabled for each resource type. These are required for proper functioning of the cluster. Admission plugins ignore resources that they are not responsible for.

In addition to the defaults, the admission chain can be extended dynamically through webhook admission plugins that call out to custom webhook servers. There are two types of webhook admission plugins: a mutating admission plugin and a validating admission plugin. The mutating admission plugin runs first and can both modify resources and validate requests. The validating admission plugin validates requests and runs after the mutating admission plugin so that modifications triggered by the mutating admission plugin can also be validated.

Calling webhook servers through a mutating admission plugin can produce side effects on resources related to the target object. In such situations, you must take steps to validate that the end result is as expected.



WARNING

Dynamic admission should be used cautiously because it impacts cluster control plane operations. When calling webhook servers through webhook admission plugins in OpenShift Container Platform 4.19, ensure that you have read the documentation fully and tested for side effects of mutations. Include steps to restore resources back to their original state prior to mutation, in the event that a request does not pass through the entire admission chain.

9.2. DEFAULT ADMISSION PLUGINS

Default validating and admission plugins are enabled in OpenShift Container Platform 4.19. These default plugins contribute to fundamental control plane functionality, such as ingress policy, cluster resource limit override and quota policy.



IMPORTANT

Do not run workloads in or share access to default projects. Default projects are reserved for running core cluster components.

The following default projects are considered highly privileged: **default**, **kube-public**, **kube-system**, **openshift**, **openshift-infra**, **openshift-node**, and other system-created projects that have the **openshift.io/run-level** label set to **0** or **1**. Functionality that relies on admission plugins, such as pod security admission, security context constraints, cluster resource quotas, and image reference resolution, does not work in highly privileged projects.

The following lists contain the default admission plugins:

Example 9.1. Validating admission plugins

- **LimitRanger**
- **ServiceAccount**
- **PodNodeSelector**
- **Priority**
- **PodTolerationRestriction**
- **OwnerReferencesPermissionEnforcement**
- **PersistentVolumeClaimResize**
- **RuntimeClass**
- **CertificateApproval**
- **CertificateSigning**
- **CertificateSubjectRestriction**
- **autoscaling.openshift.io/ManagementCPUsOverride**
- **authorization.openshift.io/RestrictSubjectBindings**
- **scheduling.openshift.io/OriginPodNodeEnvironment**
- **network.openshift.io/ExternalIPRanger**
- **network.openshift.io/RestrictedEndpointsAdmission**
- **image.openshift.io/ImagePolicy**
- **security.openshift.io/SecurityContextConstraint**
- **security.openshift.io/SCCExecRestrictions**
- **route.openshift.io/IngressAdmission**

- **config.openshift.io/ValidateAPIServer**
- **config.openshift.io/ValidateAuthentication**
- **config.openshift.io/ValidateFeatureGate**
- **config.openshift.io/ValidateConsole**
- **operator.openshift.io/ValidateDNS**
- **config.openshift.io/ValidateImage**
- **config.openshift.io/ValidateOAuth**
- **config.openshift.io/ValidateProject**
- **config.openshift.io/DenyDeleteClusterConfiguration**
- **config.openshift.io/ValidateScheduler**
- **quota.openshift.io/ValidateClusterResourceQuota**
- **security.openshift.io/ValidateSecurityContextConstraints**
- **authorization.openshift.io/ValidateRoleBindingRestriction**
- **config.openshift.io/ValidateNetwork**
- **operator.openshift.io/ValidateKubeControllerManager**
- **ValidatingAdmissionWebhook**
- **ResourceQuota**
- **quota.openshift.io/ClusterResourceQuota**

Example 9.2. Mutating admission plugins

- **NamespaceLifecycle**
- **LimitRanger**
- **ServiceAccount**
- **NodeRestriction**
- **TaintNodesByCondition**
- **PodNodeSelector**
- **Priority**
- **DefaultTolerationSeconds**
- **PodTolerationRestriction**

- **DefaultStorageClass**
- **StorageObjectInUseProtection**
- **RuntimeClass**
- **DefaultIngressClass**
- **autoscaling.openshift.io/ManagementCPUsOverride**
- **scheduling.openshift.io/OriginPodNodeEnvironment**
- **image.openshift.io/ImagePolicy**
- **security.openshift.io/SecurityContextConstraint**
- **security.openshift.io/DefaultSecurityContextConstraints**
- **MutatingAdmissionWebhook**

9.3. WEBHOOK ADMISSION PLUGINS

In addition to OpenShift Container Platform default admission plugins, dynamic admission can be implemented through webhook admission plugins that call webhook servers, to extend the functionality of the admission chain. Webhook servers are called over HTTP at defined endpoints.

There are two types of webhook admission plugins in OpenShift Container Platform:

- During the admission process, the *mutating admission plugin* can perform tasks, such as injecting affinity labels.
- At the end of the admission process, the *validating admission plugin* can be used to make sure an object is configured properly, for example ensuring affinity labels are as expected. If the validation passes, OpenShift Container Platform schedules the object as configured.

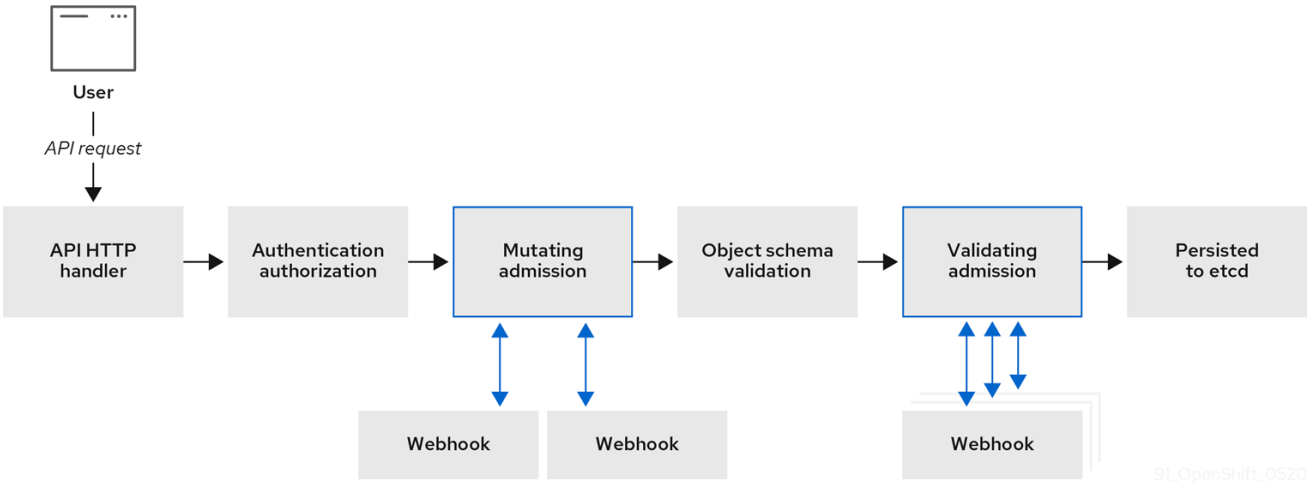
When an API request comes in, mutating or validating admission plugins use the list of external webhooks in the configuration and call them in parallel:

- If all of the webhooks approve the request, the admission chain continues.
- If any of the webhooks deny the request, the admission request is denied and the reason for doing so is based on the first denial.
- If more than one webhook denies the admission request, only the first denial reason is returned to the user.
- If an error is encountered when calling a webhook, the request is either denied or the webhook is ignored depending on the error policy set. If the error policy is set to **Ignore**, the request is unconditionally accepted in the event of a failure. If the policy is set to **Fail**, failed requests are denied. Using **Ignore** can result in unpredictable behavior for all clients.

Communication between the webhook admission plugin and the webhook server must use TLS. Generate a CA certificate and use the certificate to sign the server certificate that is used by your webhook admission server. The PEM-encoded CA certificate is supplied to the webhook admission plugin using a mechanism, such as service serving certificate secrets.

The following diagram illustrates the sequential admission chain process within which multiple webhook servers are called.

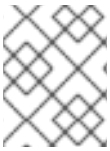
Figure 9.1. API admission chain with mutating and validating admission plugins



An example webhook admission plugin use case is where all pods must have a common set of labels. In this example, the mutating admission plugin can inject labels and the validating admission plugin can check that labels are as expected. OpenShift Container Platform would subsequently schedule pods that include required labels and reject those that do not.

Some common webhook admission plugin use cases include:

- Namespace reservation.
- Limiting custom network resources managed by the SR-IOV network device plugin.
- Defining tolerations that enable taints to qualify which pods should be scheduled on a node.
- Pod priority class validation.



NOTE

The maximum default webhook timeout value in OpenShift Container Platform is 13 seconds, and it cannot be changed.

9.4. TYPES OF WEBHOOK ADMISSION PLUGINS

Cluster administrators can call out to webhook servers through the mutating admission plugin or the validating admission plugin in the API server admission chain.

9.4.1. Mutating admission plugin

The mutating admission plugin is invoked during the mutation phase of the admission process, which allows modification of resource content before it is persisted. One example webhook that can be called through the mutating admission plugin is the Pod Node Selector feature, which uses an annotation on a namespace to find a label selector and add it to the pod specification.

Sample mutating admission plugin configuration

```
apiVersion: admissionregistration.k8s.io/v1beta1
kind: MutatingWebhookConfiguration (1)
metadata:
  name: <webhook_name> (2)
webhooks:
- name: <webhook_name> (3)
  clientConfig: (4)
    service:
      namespace: default (5)
      name: kubernetes (6)
      path: <webhook_url> (7)
    caBundle: <ca_signing_certificate> (8)
  rules: (9)
  - operations: (10)
    - <operation>
    apiGroups:
    - ""
    apiVersions:
    - "*"
    resources:
    - <resource>
  failurePolicy: <policy> (11)
  sideEffects: None
```

(1) Specifies a mutating admission plugin configuration.

(2) The name for the **MutatingWebhookConfiguration** object. Replace **<webhook_name>** with the appropriate value.

(3) The name of the webhook to call. Replace **<webhook_name>** with the appropriate value.

(4) Information about how to connect to, trust, and send data to the webhook server.

(5) The namespace where the front-end service is created.

(6) The name of the front-end service.

(7) The webhook URL used for admission requests. Replace **<webhook_url>** with the appropriate value.

(8) A PEM-encoded CA certificate that signs the server certificate that is used by the webhook server. Replace **<ca_signing_certificate>** with the appropriate certificate in base64 format.

(9) Rules that define when the API server should use this webhook admission plugin.

(10) One or more operations that trigger the API server to call this webhook admission plugin. Possible values are **create**, **update**, **delete** or **connect**. Replace **<operation>** and **<resource>** with the appropriate values.

(11) Specifies how the policy should proceed if the webhook server is unavailable. Replace **<policy>** with either **Ignore** (to unconditionally accept the request in the event of a failure) or **Fail** (to deny the failed request). Using **Ignore** can result in unpredictable behavior for all clients.



IMPORTANT

In OpenShift Container Platform 4.19, objects created by users or control loops through a mutating admission plugin might return unexpected results, especially if values set in an initial request are overwritten, which is not recommended.

9.4.2. Validating admission plugin

A validating admission plugin is invoked during the validation phase of the admission process. This phase allows the enforcement of invariants on particular API resources to ensure that the resource does not change again. The Pod Node Selector is also an example of a webhook which is called by the validating admission plugin, to ensure that all **nodeSelector** fields are constrained by the node selector restrictions on the namespace.

Sample validating admission plugin configuration

```
apiVersion: admissionregistration.k8s.io/v1beta1
kind: ValidatingWebhookConfiguration (1)
metadata:
  name: <webhook_name> (2)
webhooks:
- name: <webhook_name> (3)
  clientConfig: (4)
    service:
      namespace: default (5)
      name: kubernetes (6)
      path: <webhook_url> (7)
    caBundle: <ca_signing_certificate> (8)
  rules: (9)
  - operations: (10)
    - <operation>
    apiGroups:
    - ""
    apiVersions:
    - "*"
    resources:
    - <resource>
  failurePolicy: <policy> (11)
  sideEffects: Unknown
```

1. Specifies a validating admission plugin configuration.
2. The name for the **ValidatingWebhookConfiguration** object. Replace **<webhook_name>** with the appropriate value.
3. The name of the webhook to call. Replace **<webhook_name>** with the appropriate value.
4. Information about how to connect to, trust, and send data to the webhook server.
5. The namespace where the front-end service is created.
6. The name of the front-end service.
7. The webhook URL used for admission requests. Replace **<webhook_url>** with the appropriate value.

value.

8 A PEM-encoded CA certificate that signs the server certificate that is used by the webhook server. Replace **<ca_signing_certificate>** with the appropriate certificate in base64 format.

9 Rules that define when the API server should use this webhook admission plugin.

10 One or more operations that trigger the API server to call this webhook admission plugin. Possible values are **create**, **update**, **delete** or **connect**. Replace **<operation>** and **<resource>** with the appropriate values.

11 Specifies how the policy should proceed if the webhook server is unavailable. Replace **<policy>** with either **Ignore** (to unconditionally accept the request in the event of a failure) or **Fail** (to deny the failed request). Using **Ignore** can result in unpredictable behavior for all clients.

9.5. CONFIGURING DYNAMIC ADMISSION

This procedure outlines high-level steps to configure dynamic admission. The functionality of the admission chain is extended by configuring a webhook admission plugin to call out to a webhook server.

The webhook server is also configured as an aggregated API server. This allows other OpenShift Container Platform components to communicate with the webhook using internal credentials and facilitates testing using the **oc** command. Additionally, this enables role based access control (RBAC) into the webhook and prevents token information from other API servers from being disclosed to the webhook.

Prerequisites

- An OpenShift Container Platform account with cluster administrator access.
- The OpenShift Container Platform CLI (**oc**) installed.
- A published webhook server container image.

Procedure

1. Build a webhook server container image and make it available to the cluster using an image registry.
2. Create a local CA key and certificate and use them to sign the webhook server's certificate signing request (CSR).
3. Create a new project for webhook resources:

```
$ oc new-project my-webhook-namespace 1
```

1 Note that the webhook server might expect a specific name.

4. Define RBAC rules for the aggregated API service in a file called **rbac.yaml**:

```
apiVersion: v1
kind: List
items:
```

```
- apiVersion: rbac.authorization.k8s.io/v1 (1)
  kind: ClusterRoleBinding
  metadata:
    name: auth-delegator-my-webhook-namespace
  roleRef:
    kind: ClusterRole
    apiGroup: rbac.authorization.k8s.io
    name: system:auth-delegator
  subjects:
  - kind: ServiceAccount
    namespace: my-webhook-namespace
    name: server

- apiVersion: rbac.authorization.k8s.io/v1 (2)
  kind: ClusterRole
  metadata:
    annotations:
    name: system:openshift:online:my-webhook-server
  rules:
  - apiGroups:
    - online.openshift.io
    resources:
    - namespacereservations (3)
    verbs:
    - get
    - list
    - watch

- apiVersion: rbac.authorization.k8s.io/v1 (4)
  kind: ClusterRole
  metadata:
    name: system:openshift:online:my-webhook-requester
  rules:
  - apiGroups:
    - admission.online.openshift.io
    resources:
    - namespacereservations (5)
    verbs:
    - create

- apiVersion: rbac.authorization.k8s.io/v1 (6)
  kind: ClusterRoleBinding
  metadata:
    name: my-webhook-server-my-webhook-namespace
  roleRef:
    kind: ClusterRole
    apiGroup: rbac.authorization.k8s.io
    name: system:openshift:online:my-webhook-server
  subjects:
  - kind: ServiceAccount
    namespace: my-webhook-namespace
    name: server

- apiVersion: rbac.authorization.k8s.io/v1 (7)
```

```
  kind: RoleBinding
  metadata:
    namespace: kube-system
    name: extension-server-authentication-reader-my-webhook-namespace
  roleRef:
    kind: Role
    apiGroup: rbac.authorization.k8s.io
    name: extension-apiserver-authentication-reader
  subjects:
  - kind: ServiceAccount
    namespace: my-webhook-namespace
    name: server

- apiVersion: rbac.authorization.k8s.io/v1 8
  kind: ClusterRole
  metadata:
    name: my-cluster-role
  rules:
  - apiGroups:
    - admissionregistration.k8s.io
    resources:
    - validatingwebhookconfigurations
    - mutatingwebhookconfigurations
    verbs:
    - get
    - list
    - watch
  - apiGroups:
    - ""
    resources:
    - namespaces
    verbs:
    - get
    - list
    - watch

- apiVersion: rbac.authorization.k8s.io/v1
  kind: ClusterRoleBinding
  metadata:
    name: my-cluster-role
  roleRef:
    kind: ClusterRole
    apiGroup: rbac.authorization.k8s.io
    name: my-cluster-role
  subjects:
  - kind: ServiceAccount
    namespace: my-webhook-namespace
    name: server
```

1. Delegates authentication and authorization to the webhook server API.
2. Allows the webhook server to access cluster resources.
3. Points to resources. This example points to the **namespacereservations** resource.
4. Enables the aggregated API server to create admission reviews.

5 Points to resources. This example points to the **namespacereservations** resource.

6 Enables the webhook server to access cluster resources.

7 Role binding to read the configuration for terminating authentication.

8 Default cluster role and cluster role bindings for an aggregated API server.

5. Apply those RBAC rules to the cluster:

```
$ oc auth reconcile -f rbac.yaml
```

6. Create a YAML file called **webhook-daemonset.yaml** that is used to deploy a webhook as a daemon set server in a namespace:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  namespace: my-webhook-namespace
  name: server
  labels:
    server: "true"
spec:
  selector:
    matchLabels:
      server: "true"
  template:
    metadata:
      name: server
      labels:
        server: "true"
    spec:
      serviceAccountName: server
      containers:
      - name: my-webhook-container 1
        image: <image_registry_username>/<image_path>:<tag> 2
        imagePullPolicy: IfNotPresent
        command:
        - <container_commands> 3
        ports:
        - containerPort: 8443 4
        volumeMounts:
        - mountPath: /var/serving-cert
          name: serving-cert
        readinessProbe:
          httpGet:
            path: /healthz
            port: 8443 5
            scheme: HTTPS
      volumes:
      - name: serving-cert
        secret:
          defaultMode: 420
          secretName: server-serving-cert
```

1. Note that the webhook server might expect a specific container name.
2. Points to a webhook server container image. Replace **<image_registry_username>/<image_path>:<tag>** with the appropriate value.
3. Specifies webhook container run commands. Replace **<container_commands>** with the appropriate value.
4. Defines the target port within pods. This example uses port 8443.
5. Specifies the port used by the readiness probe. This example uses port 8443.

7. Deploy the daemon set:

```
$ oc apply -f webhook-daemonset.yaml
```

8. Define a secret for the service serving certificate signer, within a YAML file called **webhook-secret.yaml**:

```
apiVersion: v1
kind: Secret
metadata:
  namespace: my-webhook-namespace
  name: server-serving-cert
type: kubernetes.io/tls
data:
  tls.crt: <server_certificate> 1
  tls.key: <server_key> 2
```

1. References the signed webhook server certificate. Replace **<server_certificate>** with the appropriate certificate in base64 format.
2. References the signed webhook server key. Replace **<server_key>** with the appropriate key in base64 format.

9. Create the secret:

```
$ oc apply -f webhook-secret.yaml
```

10. Define a service account and service, within a YAML file called **webhook-service.yaml**:

```
apiVersion: v1
kind: List
items:

- apiVersion: v1
  kind: ServiceAccount
  metadata:
    namespace: my-webhook-namespace
    name: server

- apiVersion: v1
  kind: Service
```

```
  metadata:
    namespace: my-webhook-namespace
    name: server
    annotations:
      service.beta.openshift.io/serving-cert-secret-name: server-serving-cert
  spec:
    selector:
      server: "true"
    ports:
    - port: 443 (1)
      targetPort: 8443 (2)
```

(1) Defines the port that the service listens on. This example uses port 443.

(2) Defines the target port within pods that the service forwards connections to. This example uses port 8443.

11. Expose the webhook server within the cluster:

```
$ oc apply -f webhook-service.yaml
```

12. Define a custom resource definition for the webhook server, in a file called **webhook-crd.yaml**:

```
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
metadata:
  name: namespacereservations.online.openshift.io (1)
spec:
  group: online.openshift.io (2)
  version: v1alpha1 (3)
  scope: Cluster (4)
  names:
    plural: namespacereservations (5)
    singular: namespacereservation (6)
    kind: NamespaceReservation (7)
```

(1) Reflects **CustomResourceDefinition spec** values and is in the format **<plural>.<group>**. This example uses the **namespacereservations** resource.

(2) REST API group name.

(3) REST API version name.

(4) Accepted values are **Namespaced** or **Cluster**.

(5) Plural name to be included in URL.

(6) Alias seen in **oc** output.

(7) The reference for resource manifests.

13. Apply the custom resource definition:

```
$ oc apply -f webhook-crd.yaml
```

14. Configure the webhook server also as an aggregated API server, within a file called **webhook-api-service.yaml**:

```
apiVersion: apiregistration.k8s.io/v1beta1
kind: APIService
metadata:
  name: v1beta1.admission.online.openshift.io
spec:
  caBundle: <ca_signing_certificate> (1)
  group: admission.online.openshift.io
  groupPriorityMinimum: 1000
  versionPriority: 15
  service:
    name: server
    namespace: my-webhook-namespace
  version: v1beta1
```

(1) A PEM-encoded CA certificate that signs the server certificate that is used by the webhook server. Replace **<ca_signing_certificate>** with the appropriate certificate in base64 format.

15. Deploy the aggregated API service:

```
$ oc apply -f webhook-api-service.yaml
```

16. Define the webhook admission plugin configuration within a file called **webhook-config.yaml**. This example uses the validating admission plugin:

```
apiVersion: admissionregistration.k8s.io/v1beta1
kind: ValidatingWebhookConfiguration
metadata:
  name: namespacereservations.admission.online.openshift.io (1)
webhooks:
- name: namespacereservations.admission.online.openshift.io (2)
  clientConfig:
    service: (3)
      namespace: default
      name: kubernetes
      path: /apis/admission.online.openshift.io/v1beta1/namespacereservations (4)
    caBundle: <ca_signing_certificate> (5)
  rules:
  - operations:
    - CREATE
    apiGroups:
    - project.openshift.io
    apiVersions:
    - "*"
    resources:
    - projectrequests
  - operations:
    - CREATE
```

```
    apiGroups:
    - ""
    apiVersions:
    - "*"
    resources:
    - namespaces
  failurePolicy: Fail
```

1. Name for the **ValidatingWebhookConfiguration** object. This example uses the **namespacereservations** resource.
2. Name of the webhook to call. This example uses the **namespacereservations** resource.
3. Enables access to the webhook server through the aggregated API.
4. The webhook URL used for admission requests. This example uses the **namespacereservation** resource.
5. A PEM-encoded CA certificate that signs the server certificate that is used by the webhook server. Replace **<ca_signing_certificate>** with the appropriate certificate in base64 format.

17. Deploy the webhook:

```
$ oc apply -f webhook-config.yaml
```

18. Verify that the webhook is functioning as expected. For example, if you have configured dynamic admission to reserve specific namespaces, confirm that requests to create those namespaces are rejected and that requests to create non-reserved namespaces succeed.

9.6. ADDITIONAL RESOURCES

- Configuring the SR-IOV Network Operator
- Controlling pod placement using node taints
- Pod priority names